

向量

抽象数据类型：接口和实现

Abstract Data Type vs. Data Structure

抽象数据类型 = 数据模型 + 定义在该模型上的一组操作

抽象定义

一种定义

外部的逻辑特性

不考虑时间复杂度

操作&语义

不涉及数据的存储方式

数据结构 = 基于某种特定语言，实现ADT的一整套算法

具体实现

多种实现

内部的表示与实现

与复杂度密切相关

完整的算法

要考虑数据的具体存储机制

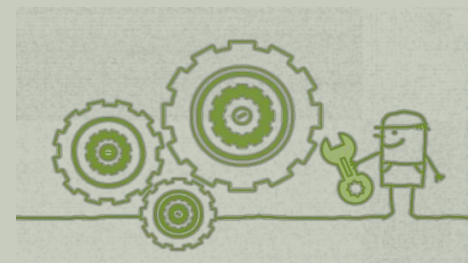


Application



Manual

Interface



Implementation

Application = Interface x Implementation

❖ 在数据结构的具体实现与实际应用之间

ADT就分工与接口制定了统一的规范

- 实现：高效兑现数据结构的ADT接口操作
//做冰箱、造汽车
- 应用：便捷地通过操作接口使用数据结构
//用冰箱、开汽车

❖ 按照ADT规范

- 高层算法设计者与底层数据结构实现者
可高效地分工协作
- 不同的算法与数据结构可以便捷组合
- 每种操作接口只需统一地实现一次
代码篇幅缩短，软件复用度提高



Application



Manual

Interface



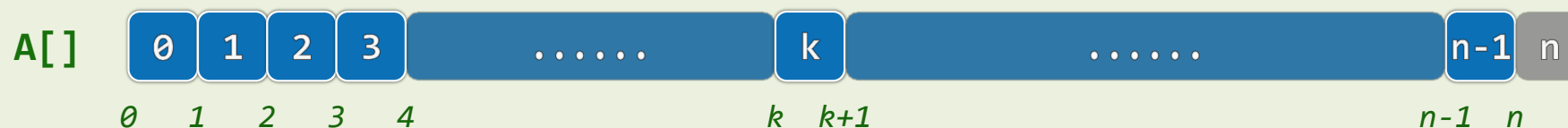
Implementation

向量

抽象数据类型：从数组到向量

循序访问

- ❖ C/C++语言中，数组元素与编号一一对应：A[0], A[1], A[2],, A[n-1]
- ❖ 反之，元素各由**编号**唯一指代，并可**直接访问** //故亦称作**线性数组** (linear array)



A[i]的物理地址 = A + i×s, s为单个元素占用的空间量

- ❖ 向量是数组的**抽象与泛化**，由一组元素按线性次序**封装**而成
 - 各元素与[0, n)内的**秩** (rank) 一一对应: `using Rank = int; //call-by-rank`
 - 操作、管理维护更加简化、统一与安全
 - 元素类型可灵活**选取**，便于**定制**复杂数据结构 `//Vector< PFCTree* > pfcForest;`

向量ADT接口

操作	功能	适用对象
size()	报告向量当前的规模（元素总数）	向量
get(r)	获取秩为r的元素	向量
put(r, e)	用e替换秩为r元素的数值	向量
insert(r, e)	e作为秩为r元素插入，原后继依次后移	向量
remove(r)	删除秩为r的元素，返回该元素原值	向量
disordered()	判断所有元素是否已按非降序排列	向量
sort()	调整各元素的位置，使之按非降序排列	向量
find(e)	查找目标元素e	向量
search(e)	查找e，返回不大于e且秩最大的元素	有序向量
deduplicate(), unify()	剔除重复元素	向量/有序向量
traverse()	遍历向量并统一处理所有元素	向量

ADT操作实例

操作	输出	向量组成 (自左向右)
初始化		
insert(0, 9)		9
insert(0, 4)		4 9
insert(1, 5)		4 5 9
put(1, 2)		4 2 9
get(2)	9	4 2 9
insert(3, 6)		4 2 9 6
insert(1, 7)		4 7 2 9 6
remove(2)	2	4 7 9 6
insert(1, 3)		4 3 7 9 6
insert(3, 4)		4 3 7 4 9 6
size()	6	4 3 7 4 9 6

操作	输出	向量组成 (自左向右)
disordered()	3	4 3 7 4 9 6
find(9)	4	4 3 7 4 9 6
find(5)	-1	4 3 7 4 9 6
sort()		3 4 4 6 7 9
disordered()	0	3 4 4 6 7 9
search(1)	-1	3 4 4 6 7 9
search(4)	2	3 4 4 6 7 9
search(8)	4	3 4 4 6 7 9
search(9)	5	3 4 4 6 7 9
search(10)	5	3 4 4 6 7 9
uniquify()		3 4 6 7 9
search(9)	4	3 4 6 7 9

向量

抽象数据类型：模版类

```
template <typename T> class Vector { //向量模板类
```

```
private: Rank _size; Rank _capacity; T* _elem; //规模、容量、数据区
```

```
protected:
```

```
    /* ... 内部函数 */
```

```
public:
```

```
    /* ... 构造函数 */
```

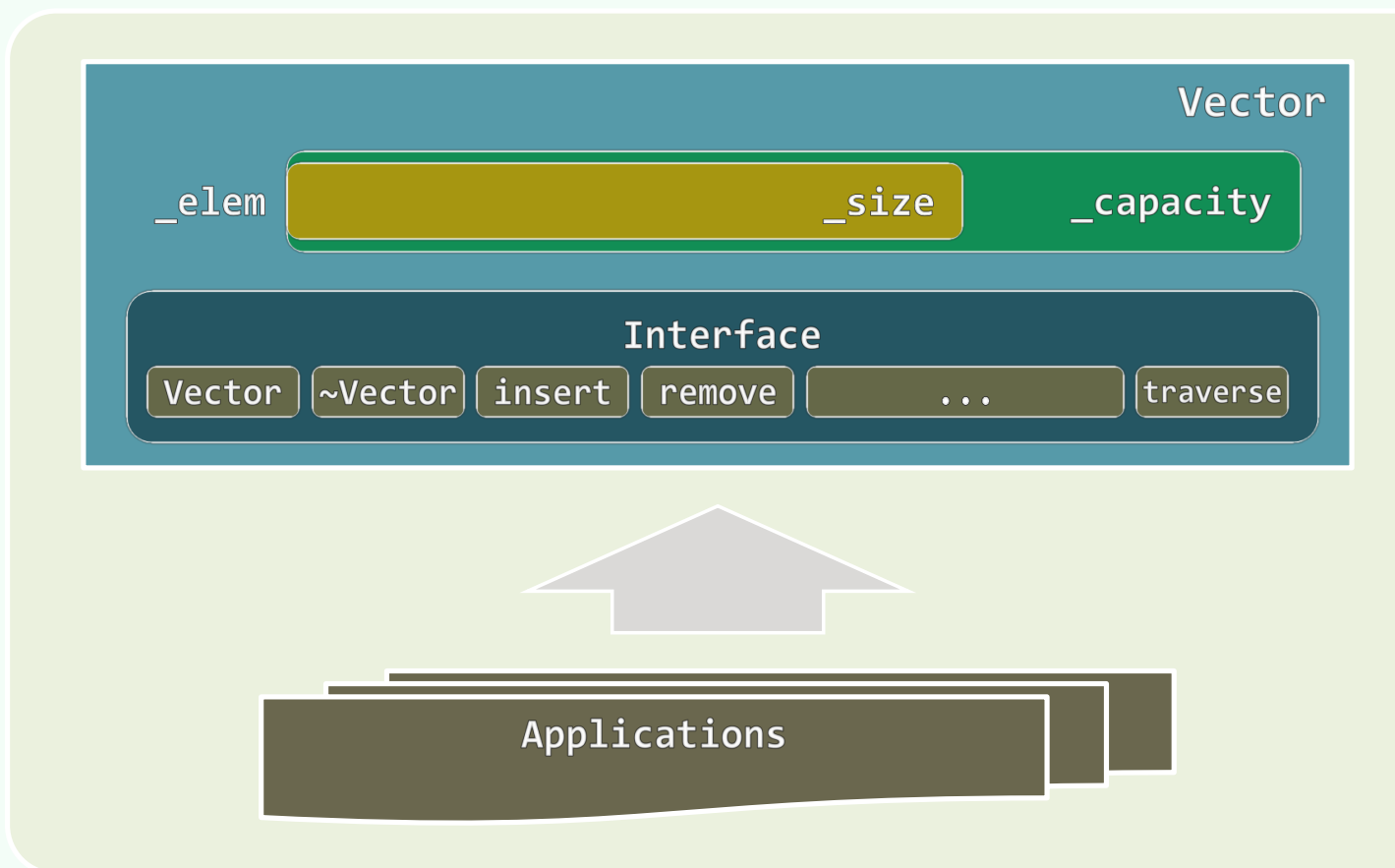
```
    /* ... 析构函数 */
```

```
    /* ... 只读接口 */
```

```
    /* ... 可写接口 */
```

```
    /* ... 遍历接口 */
```

```
};
```



构造 + 析构：重载

```
#define DEFAULT_CAPACITY 3 //默认初始容量 (实际应用中可设置为更大)
```

```
Vector( int c = DEFAULT_CAPACITY )
```

```
{ _elem = new T[ _capacity = c ]; _size = 0; } //默认
```

```
Vector( T const * A, Rank lo, Rank hi ) //数组区间复制
```

```
{ copyFrom( A, lo, hi ); }
```

```
Vector( Vector<T> const & V, Rank lo, Rank hi ) //向量区间复制
```

```
{ copyFrom( V._elem, lo, hi ); }
```

```
Vector( Vector<T> const & V ) //向量整体复制
```

```
{ copyFrom( V._elem, 0, V._size ); }
```

```
~Vector() { delete [] _elem; } //释放内部空间
```

基于复制的构造

```
template <typename T> //T为基本类型, 或已重载赋值操作符 '='
```

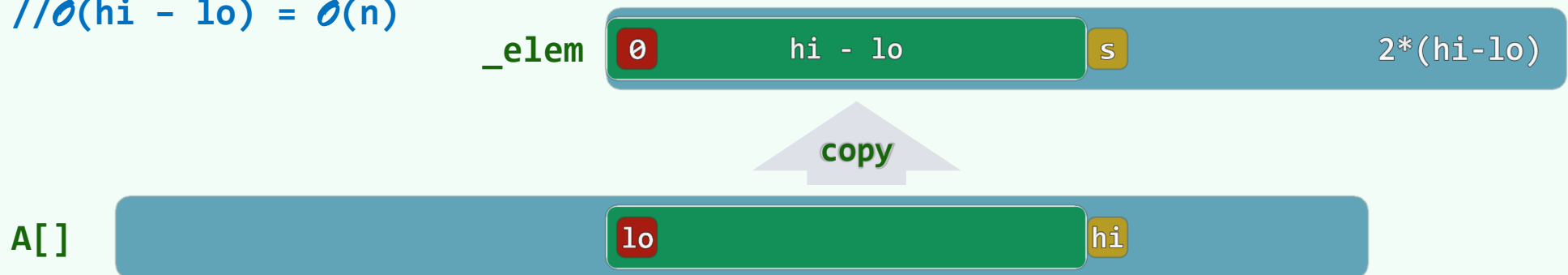
```
void Vector<T>::copyFrom( T const * A, Rank lo, Rank hi ) { //A中元素不致被篡改
```

```
    _elem = new T[ _capacity = max( DEFAULT_CAPACITY, 2*(hi - lo) ) ]; //分配空间
```

```
    for ( _size = 0; lo < hi; _size++, lo++ ) //A[lo, hi)内的元素, 逐一
```

```
        _elem[ _size ] = A[ lo ]; //复制至_elem[0, hi-lo)
```

```
} //O(hi - lo) = O(n)
```



向量

无序向量：基本操作

元素访问

❖ `V.get(r)`和`V.put(r,e)`不够便捷、直观，可否沿用数组的访问方式`V[r]`？

可以！比如，通过**重载**下标操作符“`[]`”

❖ `template <typename T> //可作为左值: V[r] = (T) (2*x + 3)`

```
T & Vector<T>::operator[] ( Rank r ) { return _elem[ r ]; }
```

❖ `template <typename T> //仅限于右值: T x = V[r] + U[s] * W[t]`

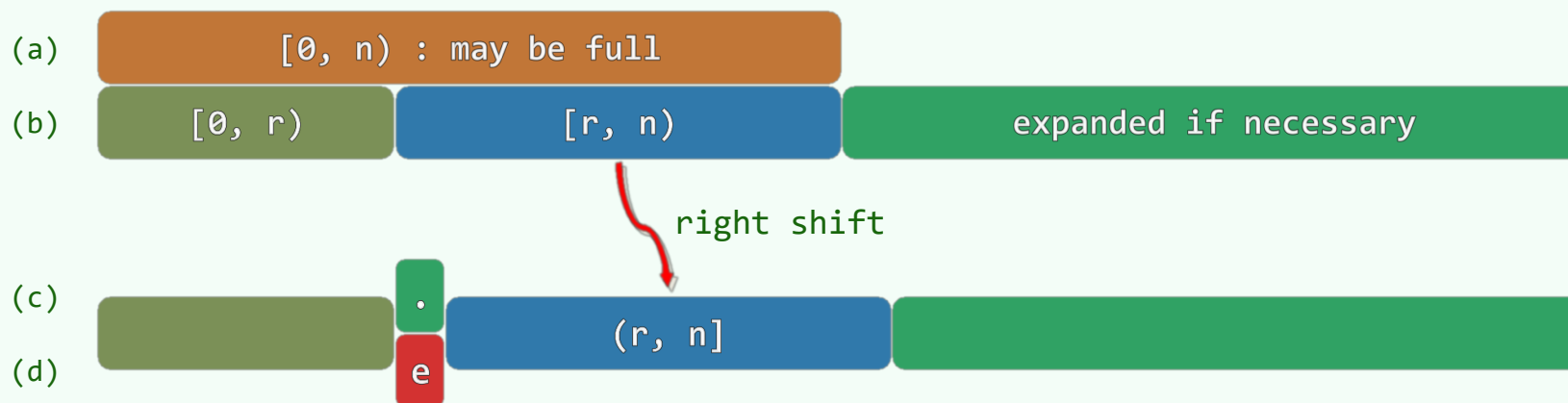
```
const T & Vector<T>::operator[] ( Rank r ) const { return _elem[ r ]; }
```

❖ 这里采用了简易的方式处理意外和错误（比如，入口参数约定： $0 \leq r < _size$ ）

实际应用中，应采用更为严格的方式

插入

```
template <typename T> Rank Vector<T>::insert( Rank r, T const & e ) { //0<=r<=size
    expand(); //如必要, 先扩容
    for ( Rank i = _size; r < i; i-- ) //0(n-r): 自后向前    为什么要从后向前移动?
        _elem[i] = _elem[i - 1]; //后继元素顺次后移一个单元
    _elem[r] = e; _size++; return r; //置入新元素, 更新容量, 返回秩
}
```

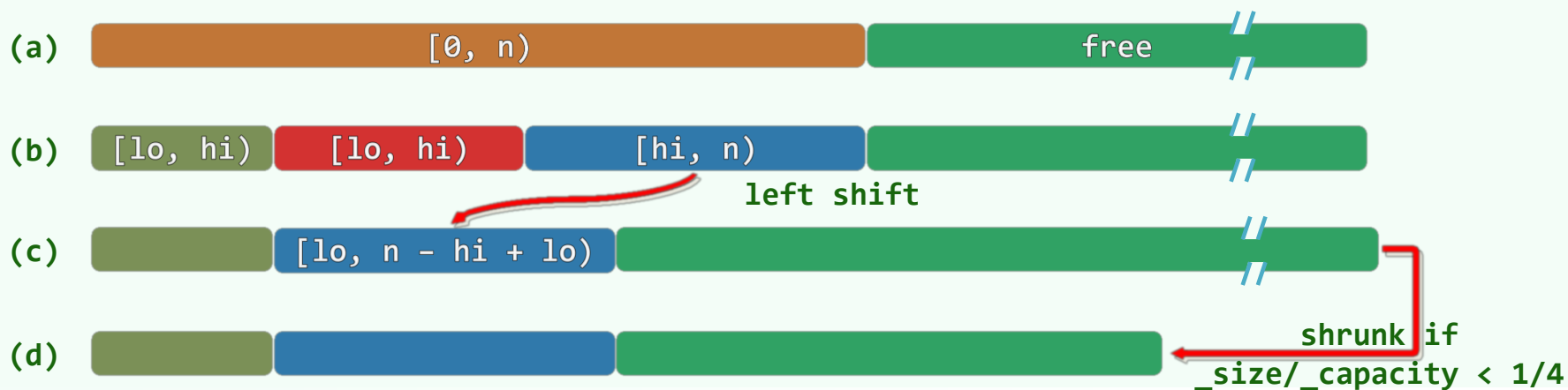


区间删除

```
template <typename T> int Vector<T>::remove( Rank lo, Rank hi ) { //0<=lo<=hi<=n
    if ( lo == hi ) return 0; //出于效率考虑, 单独处理退化情况
    while ( hi < _size ) _elem[ lo ++ ] = _elem[ hi ++ ]; //O(n-hi): [hi,n)顺次前移
    _size = lo; shrink(); //更新规模, 若有必要则缩容
    return hi - lo; //返回被删除元素的数目
}
```

为什么要从前向后移动?

何时出现这类问题? 试举一例



单元素删除

```
template <typename T>

T Vector<T>::remove( Rank r ) {

    T e = _elem[r]; //备份

    remove( r, r+1 ); // “区间” 删除

    return e; //返回被删除元素

} //O(n-r), 0 <= r < size
```

❖ 也就是将单元素视作区间的特例：

$$[r] = [r, r + 1)$$

❖ 反过来，通过反复调用remove(r)接口实现remove(lo, hi)呢？

❖ 每次循环耗时，正比于删除区间的后缀长度

$$n - hi = O(n)$$

而循环次数等于区间宽度

$$hi - lo = O(n)$$

如此，将导致总体 $O(n^2)$ 的复杂度

向量
无序向量：查找

无序向量：判等器

```
❖ template <typename K, typename V> struct Entry { //词条模板类

    K key; V value; //关键码、数值

    Entry ( K k = K(), V v = V() ) : key ( k ), value ( v ) {}; //默认构造函数

    Entry ( Entry<K, V> const& e ) : key ( e.key ), value ( e.value ) {}; //克隆

    bool operator== ( Entry<K, V> const& e ) { return key == e.key; } //等于

    bool operator!= ( Entry<K, V> const& e ) { return key != e.key; } //不等

    /* ... */

};
```

有序向量：比较器

```
❖ template <typename K, typename V> struct Entry { //词条模板类  
    K key; V value; //关键码、数值  
  
    Entry ( K k = K(), V v = V() ) : key ( k ), value ( v ) {}; //默认构造函数  
  
    Entry ( Entry<K, V> const& e ) : key ( e.key ), value ( e.value ) {}; //克隆  
  
    bool operator== ( Entry<K, V> const& e ) { return key == e.key; } //等于  
  
    bool operator!= ( Entry<K, V> const& e ) { return key != e.key; } //不等于  
  
    bool operator< ( Entry<K, V> const& e ) { return key < e.key; } //小于  
  
    bool operator> ( Entry<K, V> const& e ) { return key > e.key; } //大于  
}; //得益于比较器和判等器，从此往后，不必严格区分词条及其对应的关键码
```

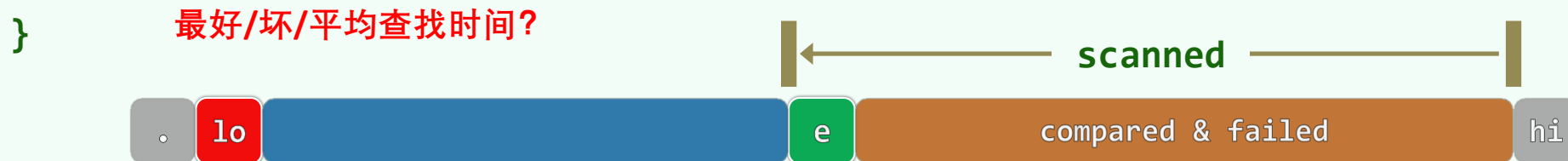
顺序查找

❖ `template <typename T> Rank Vector<T>:: //  $\mathcal{O}(hi - lo) = \mathcal{O}(n)$`

`find( T const & e, Rank lo, Rank hi ) const { //  $0 \leq lo < hi \leq \_size$`

`while ( (lo < hi--) && (e != _elem[hi]) ); // 逆向查找`

`return hi; // 返回值小于lo即意味着失败; 否则即命中者的秩 (有多个时, 返回最大者)`



❖ 输入敏感 (input-sensitive) : 最好 $\mathcal{O}(1)$, 最差 $\mathcal{O}(n)$

向量
无序向量：去重

Vector::deduplicate()

```
template <typename T> Rank Vector<T>::deduplicate() {
```

```
    Rank oldSize = _size;
```

```
    for ( Rank i = 1; i < _size; )
```



```
        if ( find( _elem[i], 0, i ) < 0 )
```

```
            i++;
```



```
        else
```

```
            remove(i);
```



```
    return oldSize - _size;
```

时间复杂度?

```
}
```

唯一化：正确性

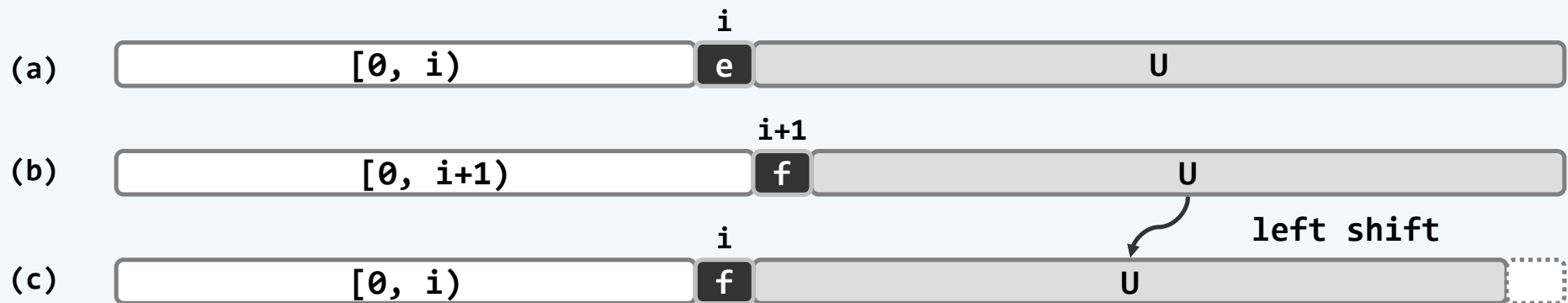
❖ **不变性**：在当前元素 $V[i]$ 的前缀 $V[0, i)$ 中，各元素彼此**互异**
初始 $i = 1$ 时自然成立；其余的一般情况，...

❖ **单调性**：随着反复的while迭代

1) 当前元素**前缀**的长度单调非降，且迟早增至`_size` //1)和2)对应

2) 当前元素**后缀**的长度单调下降，且迟早减至0 //2)更易把握

故算法**必然终止**，且至多迭代 $O(n)$ 轮



向量

无序向量：遍历

遍历

❖ 对向量中的每一元素，统一实施visit()操作 //如何指定visit()? 如何将其传递到向量内部?

❖ template <typename T> //函数指针，只读或局部性修改

```
void Vector<T>::traverse( void ( * visit )( T & ) )  
  
    { for ( Rank i = 0; i < _size; i++ ) visit( _elem[i] ); }
```

❖ template <typename T> template <typename VST> //函数对象，全局性修改更便捷

```
void Vector<T>::traverse( VST & visit )  
  
    { for ( Rank i = 0; i < _size; i++ ) visit( _elem[i] ); }
```

实例

❖ 比如，为统一地将向量中所有元素分别**加一**，只需

- 实现一个可使单个T类型元素加一的类（结构）

```
template <typename T> //假设T可直接递增或已重载操作符 “++”  
  
struct Increase //函数对象：通过重载操作符 “()” 实现  
  
    {    virtual void operator()( T & e ) { e++; }    }; //加一
```

- 将其作为参数传递给遍历算法

```
template <typename T> void increase( Vector<T> & V )  
  
    {    V.traverse( Increase<T>() );    } //即可以之作为基本操作，遍历向量
```

❖ 作为练习，可模仿此例，实现统一的**减一**、**加倍**、**求和**等遍历功能

向量
有序向量：唯一化

有序性及其甄别

❖ 与起泡排序算法的理解相同：**有序**/**无序**序列中，**任意**/**总有**一对相邻元素**顺序**/**逆序**

❖ 因此，相邻逆序对的数目，可用以**度量**向量的**逆序**程度

❖ `template <typename T>`

```
int Vector<T>::disordered() const { //统计向量中的逆序相邻元素对
```

```
    int n = 0; //计数器
```

```
    for ( int i = 1; i < _size; i++ ) //逐一检查各对相邻元素
```

```
        n += ( _elem[i - 1] > _elem[i] ); //逆序则计数
```

```
    return n; //向量有序当且仅当n = 0
```

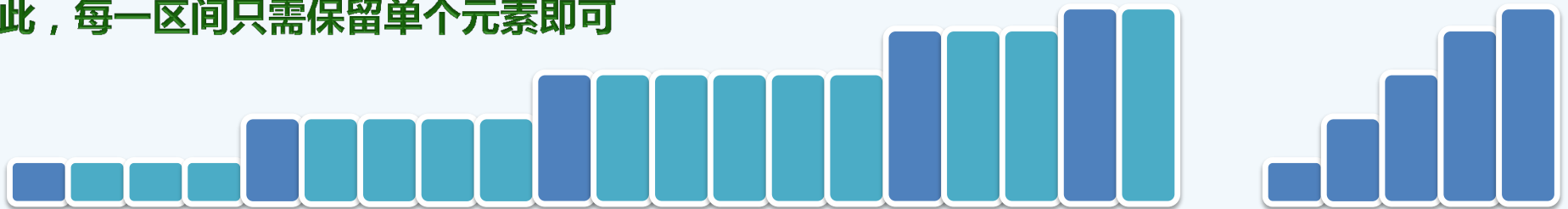
```
} //若只需判断是否有序，则首次遇到逆序对之后，即可立即终止
```

❖ 无序向量经预处理转换为有序向量之后，相关算法多可优化

低效算法

❖ 观察：在有序向量中，重复的元素必然相互紧邻构成一个区间

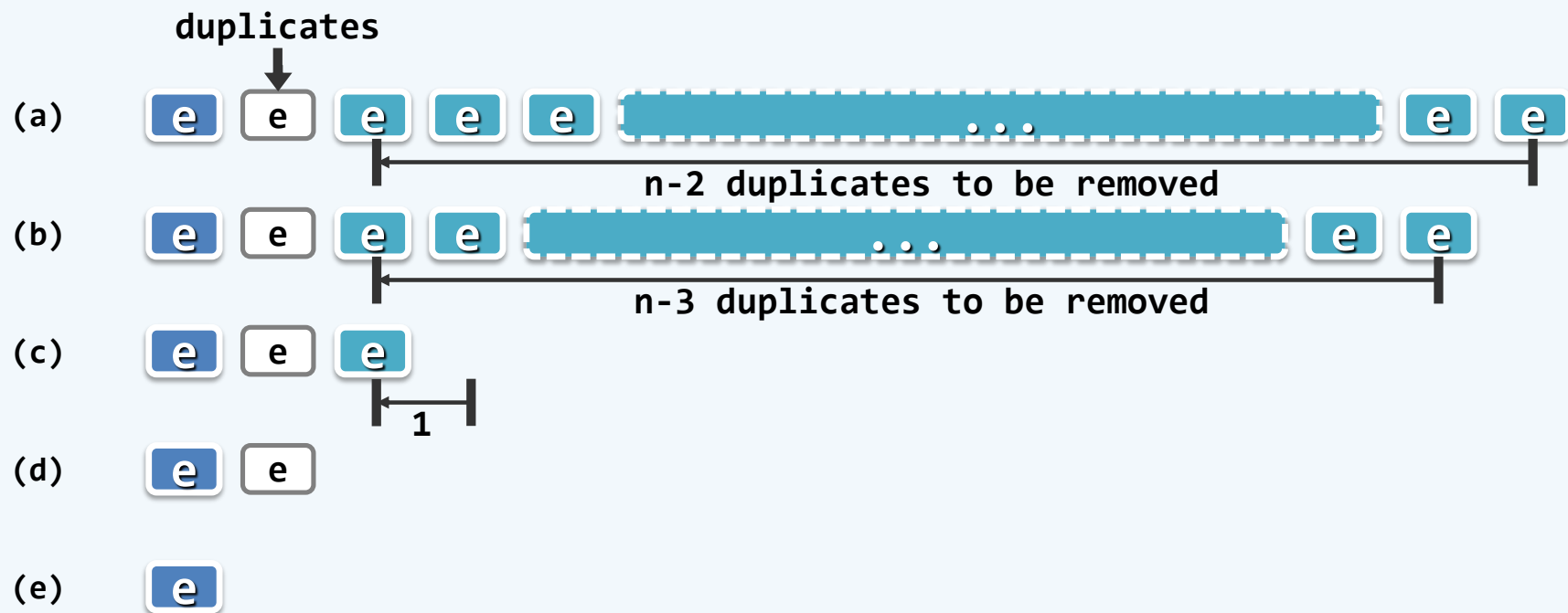
因此，每一区间只需保留单个元素即可



```
❖ template <typename T> int Vector<T>::uniquify() {  
    int oldSize = _size; int i = 1; //从首元素开始  
    while ( i < _size ) //从前向后，逐一比对各对相邻元素  
        //若雷同，则删除后者；否则，转至后一元素  
        _elem[i - 1] == _elem[i] ? remove( i ) : i++;  
    return oldSize - _size; //向量规模变化量，即删除元素总数  
} //注意：其中_size的减小，由remove()隐式地完成
```

低效算法：复杂度

- ❖ 运行时间主要取决于while循环，次数共计： $_size - 1 = n - 1$
- ❖ 最坏情况下：每次都需调用remove()，耗时 $O(n-1) \sim O(1)$ ；累计 $O(n^2)$
——尽管省去find()，总体竟与无序向量的deduplicate()相同



高效算法

❖ 反思：低效的根源在于，同一元素可作为被删除元素的后继多次前移

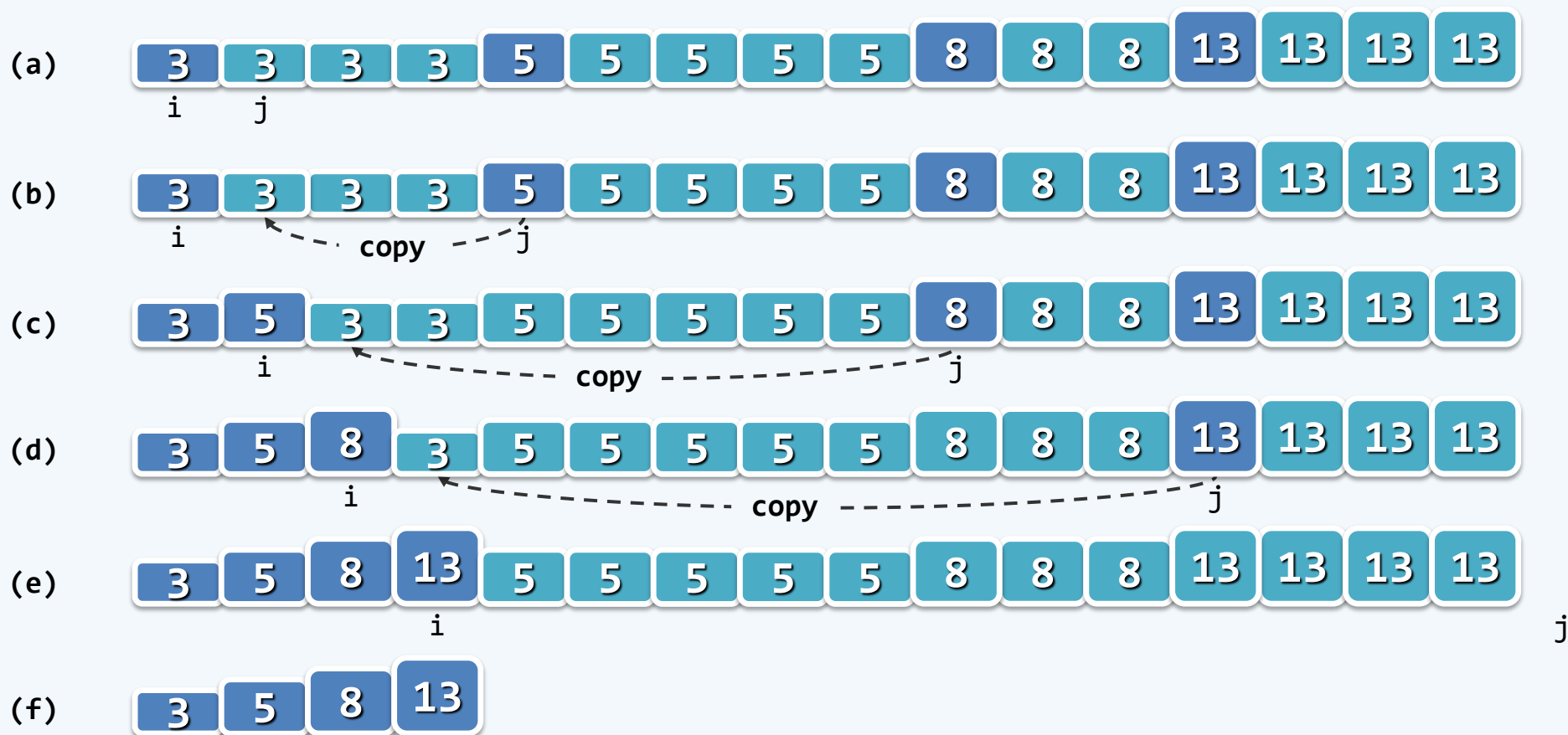
启示：若能以重复区间为单位，成批删除雷同元素，性能必将改进

```
❖ template <typename T> int Vector<T>::uniquify() {  
    Rank i = 0, j = 0; //各对互异“相邻”元素的秩  
    while ( ++ j < _size ) //逐一扫描，直至末元素  
        //跳过雷同者；发现不同元素时，向前移至紧邻于前者右侧  
        if ( _elem[ i ] != _elem[ j ] ) _elem[ ++ i ] = _elem[ j ];  
    _size = ++i; shrink(); //直接截除尾部多余元素  
    return j - i; //向量规模变化量，即被删除元素总数  
} //注意：通过remove(lo, hi)批量删除，依然不能达到高效率
```



高效算法：实例与复杂度

❖ 共计 $n - 1$ 次迭代，每次常数时间，累计 $O(n)$ 时间



向量

有序向量：二分查找（版本A）

统一接口

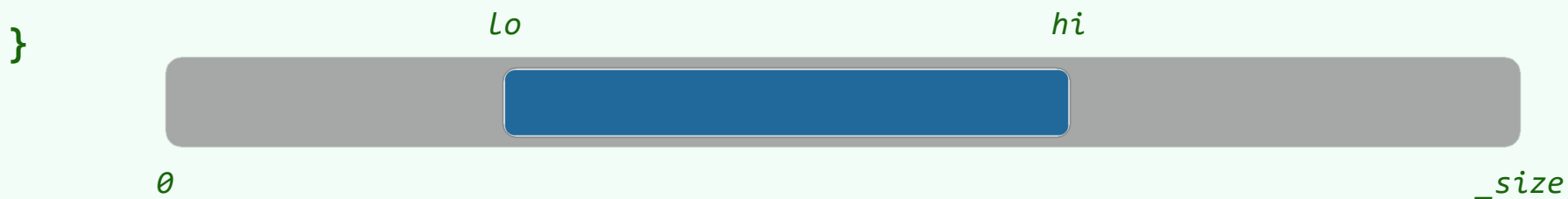
❖ `template <typename T> //查找算法统一接口, 0 <= lo < hi <= _size`

`Rank Vector<T>::search( T const & e, Rank lo, Rank hi ) const {`

`return ( rand() % 2 ) ? //按各50%的概率随机选用`

`binSearch( _elem, e, lo, hi ) //二分查找算法, 或者`

`: fibSearch( _elem, e, lo, hi ); //Fibonacci查找算法`



有序向量中，每个元素都是轴点

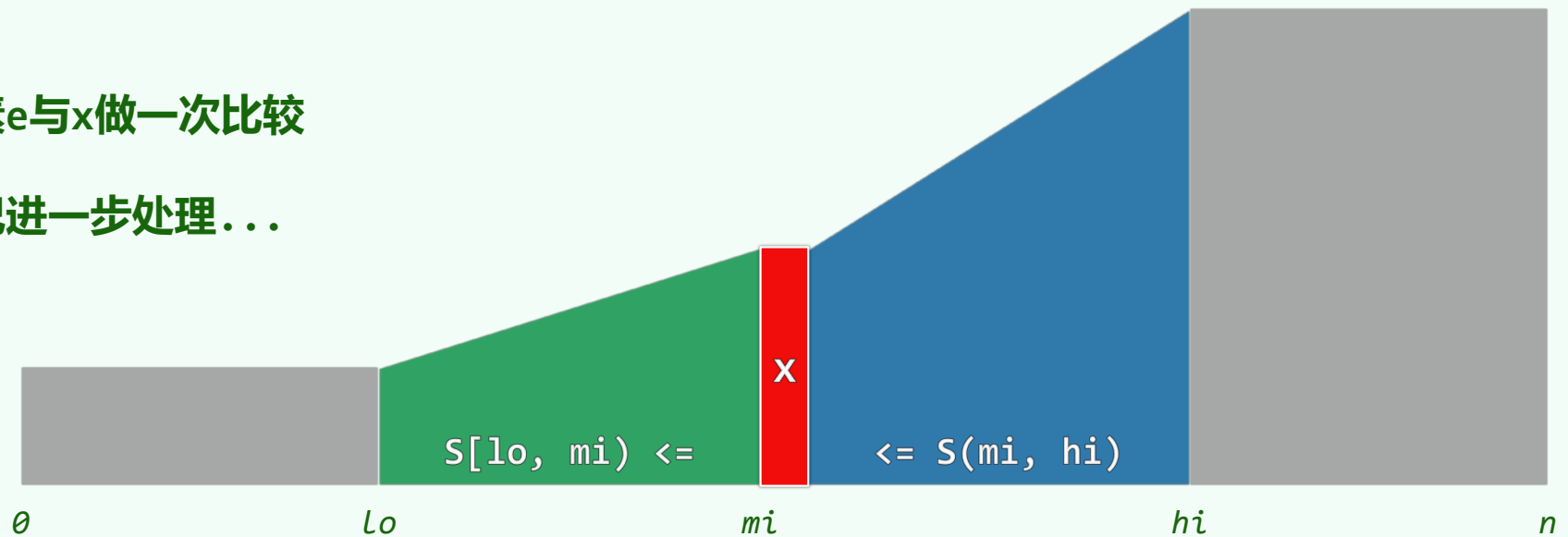
❖ 以任一元素 $x = S[mi]$ 为界，都可将待查找区间 $[lo, hi)$ 分为三部分

$$S[lo, mi) \leq S[mi] \leq S(mi, hi)$$

因此...

❖ 只需将目标元素 e 与 x 做一次比较

即可分三种情况进一步处理...



减而治之

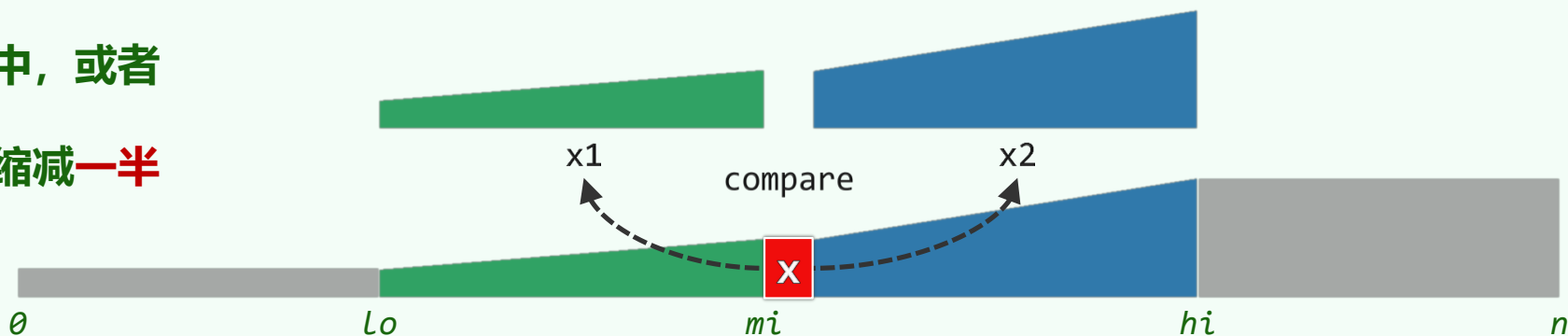
❖ 只需将目标元素 e 与 x 做一比较，即可分三种情况进一步处理：

- $e < x$ ：则 e 若存在必属于左侧子区间，故可（减除 $S[mi, hi]$ 并）递归深入 $S[lo, mi]$
- $x < e$ ：则 e 若存在必属于右侧子区间，亦可（减除 $S[lo, mi]$ 并）递归深入 $S(mi, hi)$
- $e = x$ ：已在此处命中，可随即返回 //若有多，返回何者？

❖ 若轴点 mi 取作中点，则每经过至多两次比较

❖ 或者能够命中，或者

将问题规模缩减一半



实现

❖ template <typename T> //在有序向量区间[lo, hi)内查找元素e

```
static Rank binSearch( T * S, T const & e, Rank lo, Rank hi ) {
```

```
    while ( lo < hi ) { //每步迭代可能要做两次比较判断, 有三个分支
```

```
        Rank mi = ( lo + hi ) >> 1; //轴点居中 (区间宽度折半, 等效于其数值右移一位)
```

```
        if ( e < S[mi] ) hi = mi; //深入前半段[lo, mi)
```

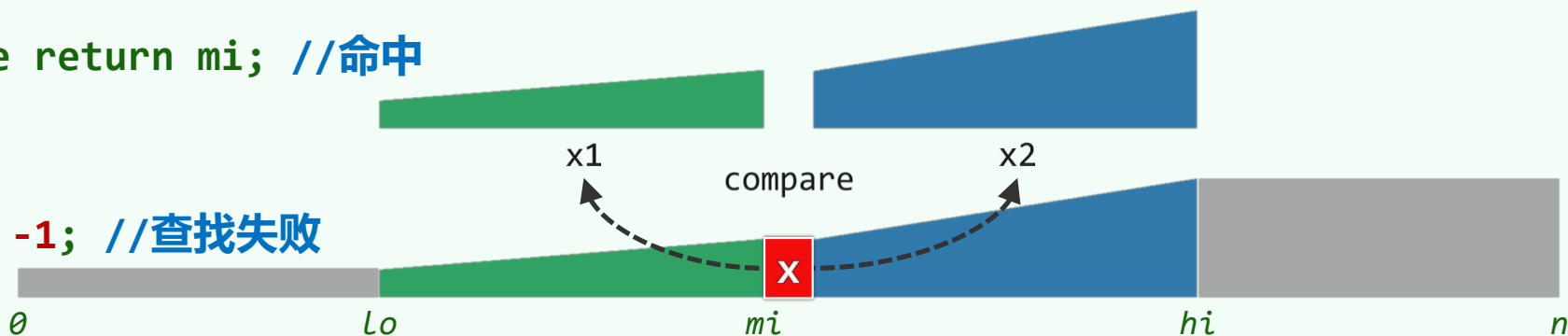
```
        else if ( S[mi] < e ) lo = mi + 1; //深入后半段(mi, hi)
```

```
        else return mi; //命中
```

```
    }
```

```
    return -1; //查找失败
```

```
}
```



实例 + 复杂度

S.search(8, 0, 7):

经 $2 + 1 + 2 = 5$ 次比较, 在S[4]命中

	0	1	2	3	4	5	6	7
(1)	2	4	5	7	8	9	12	
(2)					8	9	12	
(3)					8			

S.search(3, 0, 7):

经 $1 + 1 + 2 = 4$ 次比较, 在S[1]失败

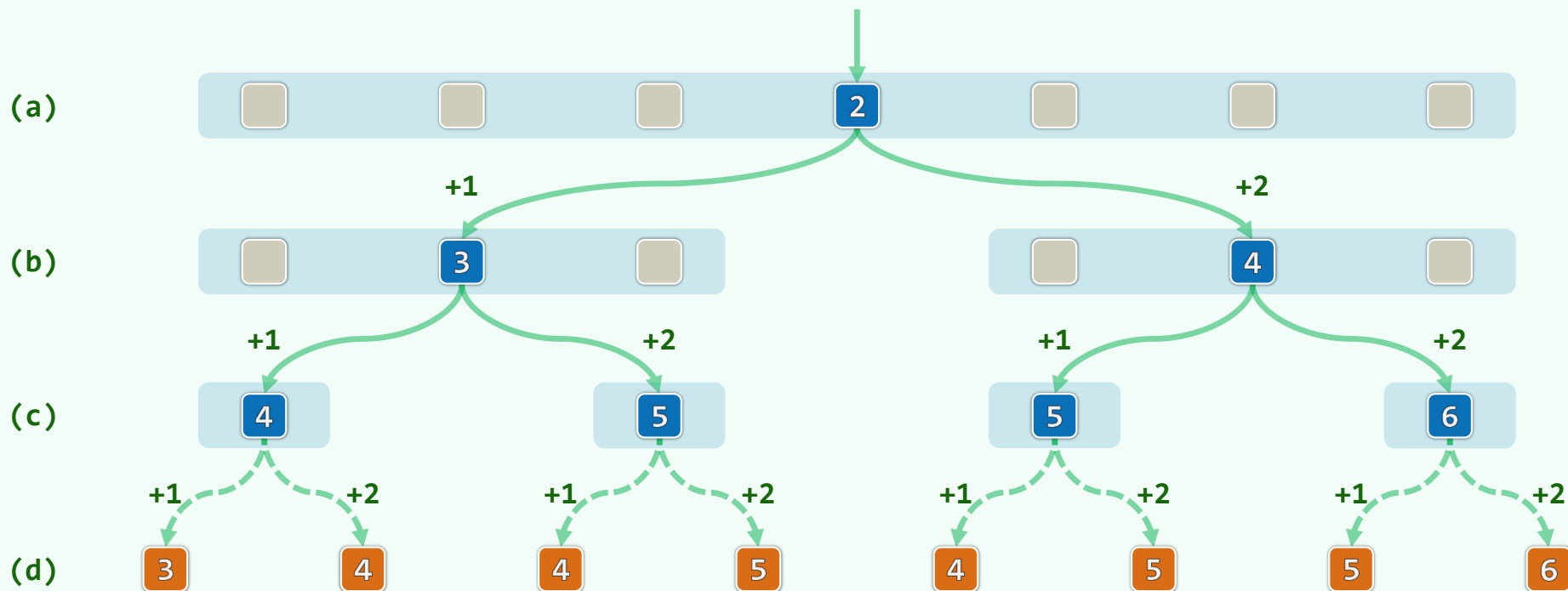
	0	1	2	3	4	5	6	7
(1)	2	4	5	7	8	9	12	
(2)	2	4	5					
(3)	2							
(4)								

❖ 线性递归: $T(n) = T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$, 大大优于顺序查找

递归跟踪: 轴点总能取到中点, 递归深度 $\mathcal{O}(\log n)$; 各递归实例仅耗时 $\mathcal{O}(1)$

查找长度

- ❖ 如何更精细地评估查找算法的性能？考查关键码的比较次数 (search length)
- ❖ 通常，需分别针对**成功**与**失败**查找，从**最好**、**最坏**、**平均**等角度评估
- ❖ 比如，成功、失败时的平均查找长度均大致为 $O(1.50 \cdot \log n)$ //详见教材、习题解析



查找长度

❖ $n = 7$ 时, 各元素对应的成功查找长度为

$\{ 4, 3, 5, 2, 5, 4, 6 \}$

在等概率情况下, 平均**成功**查找长度

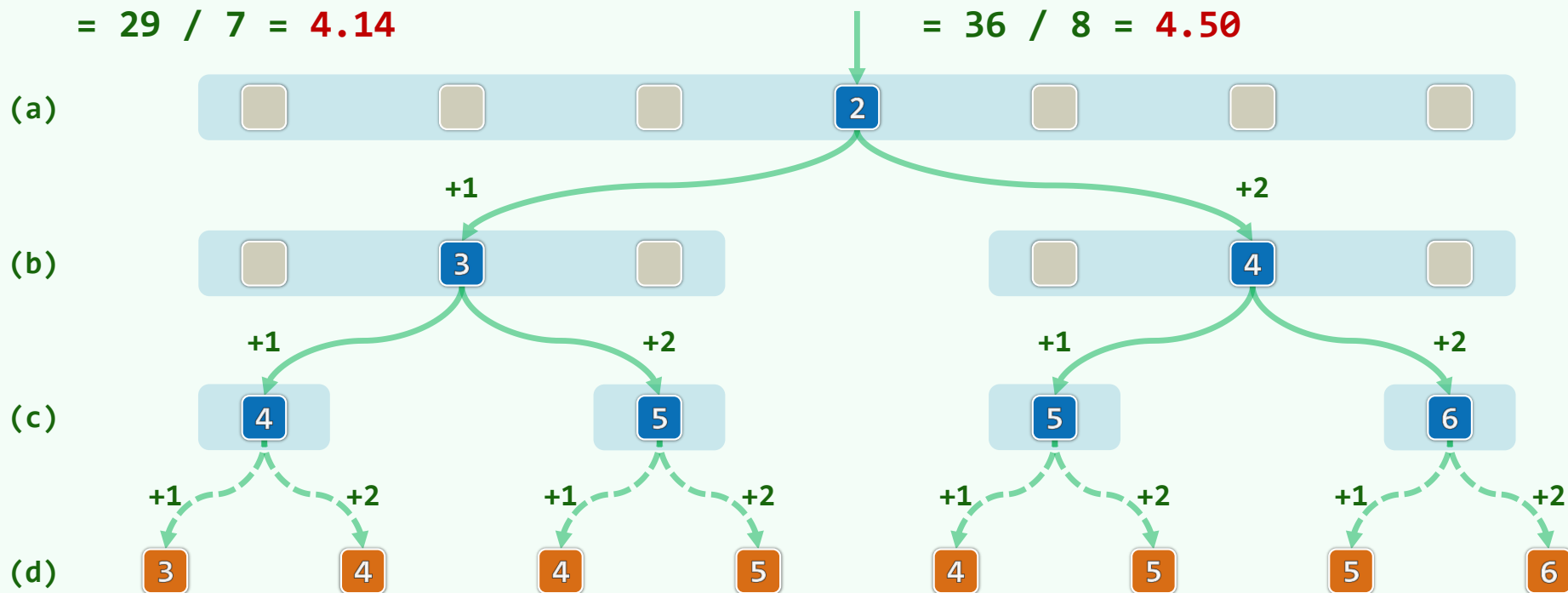
$$= 29 / 7 = 4.14$$

❖ 共8种失败情况, 查找长度分别为

$\{ 3, 4, 4, 5, 4, 5, 5, 6 \}$

在等概率情况下, 平均**失败**查找长度

$$= 36 / 8 = 4.50$$



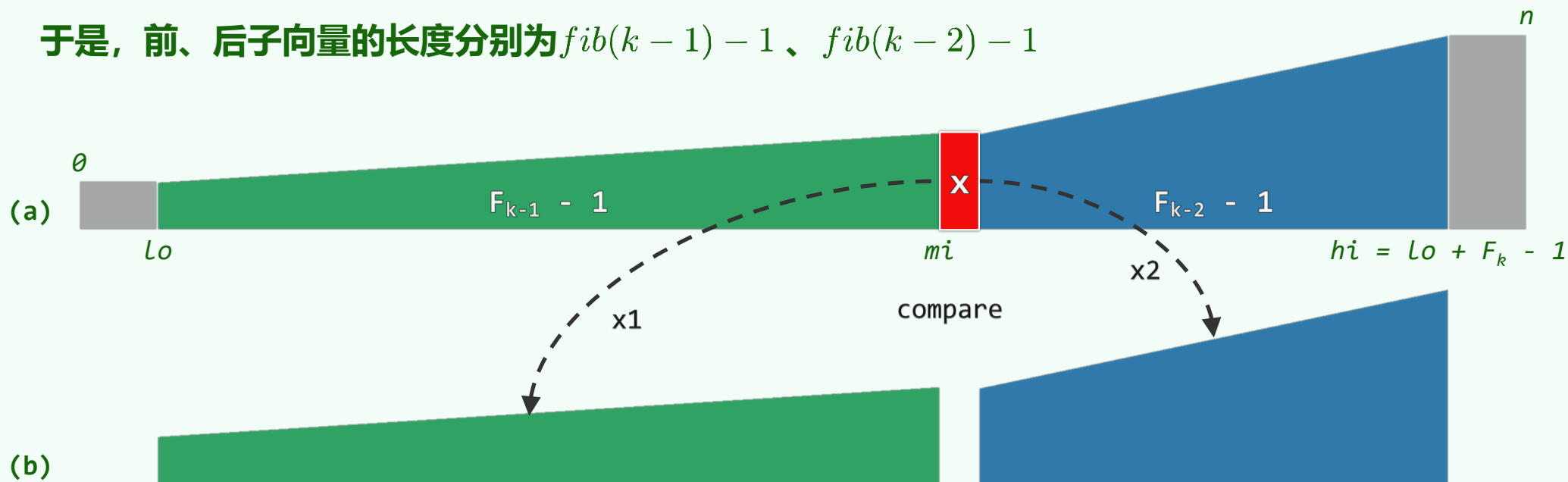
向量

有序向量: Fibonacci查找

思路及原理

- ❖ 版本A: 转向左、右分支前的关键码比较次数不等, 而递归深度却相同
- ❖ 通过递归深度的不均衡对转向成本的不均衡做补偿, 平均查找长度应能进一步缩短!
- ❖ 比如, 若有 $n = \text{fib}(k) - 1$, 则可取 $mi = \text{fib}(k - 1) - 1$

于是, 前、后子向量的长度分别为 $\text{fib}(k - 1) - 1$ 、 $\text{fib}(k - 2) - 1$



实现

```
template <typename T> //0 <= lo <= hi <= _size
static Rank fibSearch( T * S, T const & e, Rank lo, Rank hi ) {
    for ( Fib fib(hi - lo); lo < hi; ) { //Fib数列制表备查
        while ( hi - lo < fib.get() ) fib.prev(); //自后向前顺序查找轴点 (分摊 $O(1)$ )
        Rank mi = lo + fib.get() - 1; //确定形如Fib(k) - 1的轴点
        if ( e < S[mi] ) hi = mi; //深入前半段[lo, mi)
        else if ( S[mi] < e ) lo = mi + 1; //深入后半段(mi, hi)
        else return mi; //命中
    }
    return -1; //查找失败
}
```

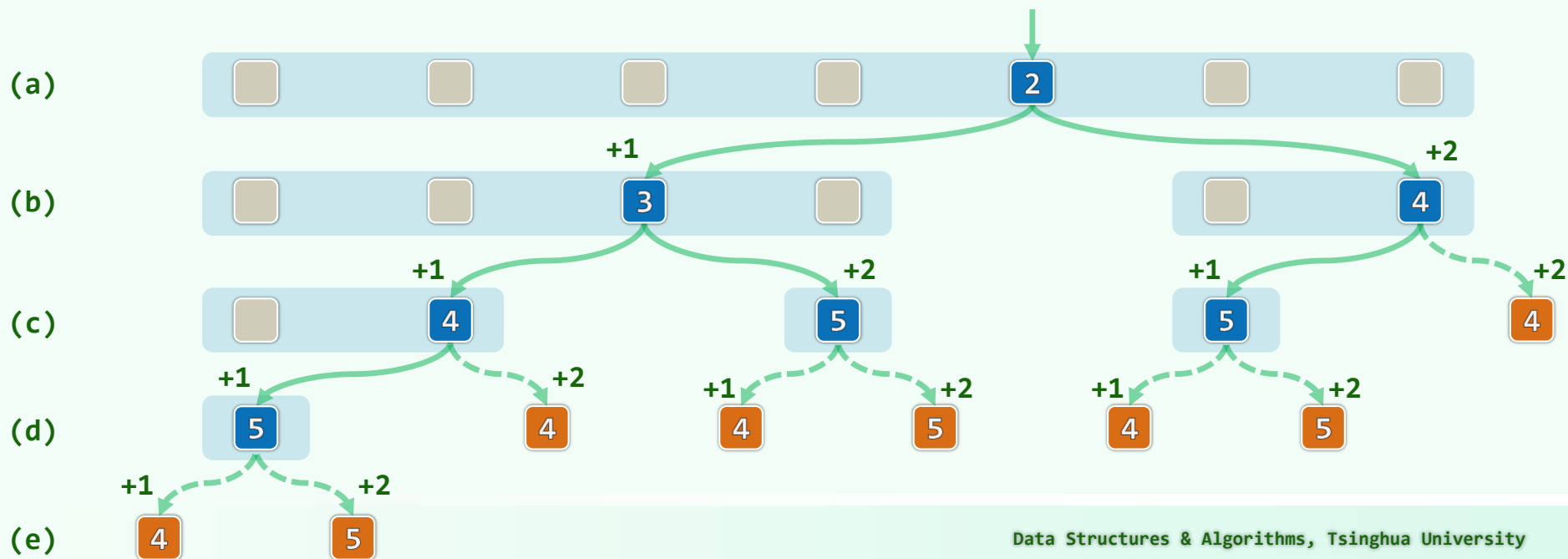
查找长度

❖ Fibonacci查找的ASL, (在常系数的意义上) 优于二分查找 //详见教材、习题解析

❖ 仍以 $n = \text{fib}(6) - 1 = 7$ 为例, 在等概率情况下

- 平均成功查找长度 = $(5 + 4 + 3 + 5 + 2 + 5 + 4) / 7 = 28/7 = 4.00$

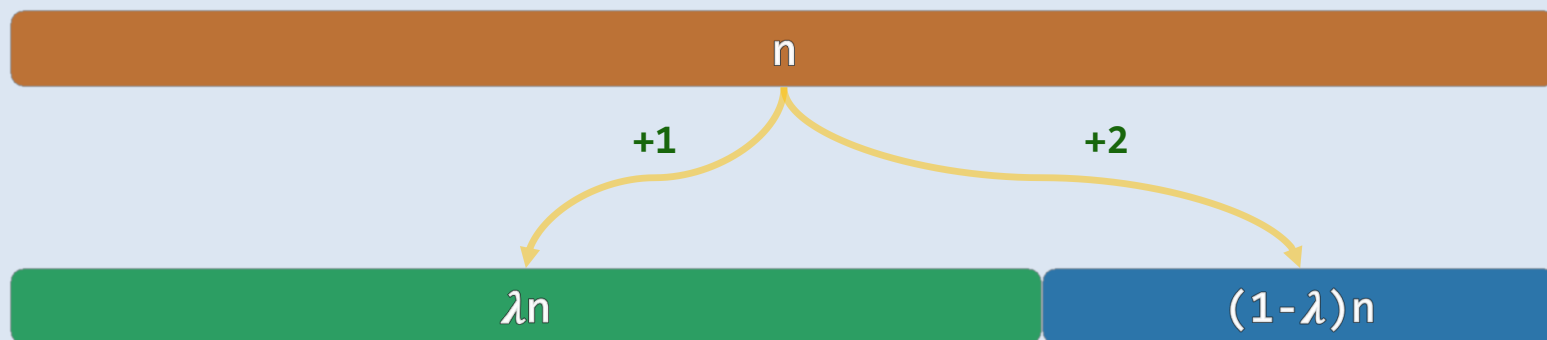
- 平均失败查找长度 = $(4 + 5 + 4 + 4 + 5 + 4 + 5 + 4) / 8 = 35 / 8 = 4.38$



通用策略

❖ 在任何区间 $[0, n)$ 内，总是选取 $[\lambda \cdot n]$ 作为轴点， $0 \leq \lambda < 1$

比如：二分查找对应于 $\lambda = 0.5$ ，Fibonacci查找对应于 $\lambda = \phi = 0.6180339\dots$

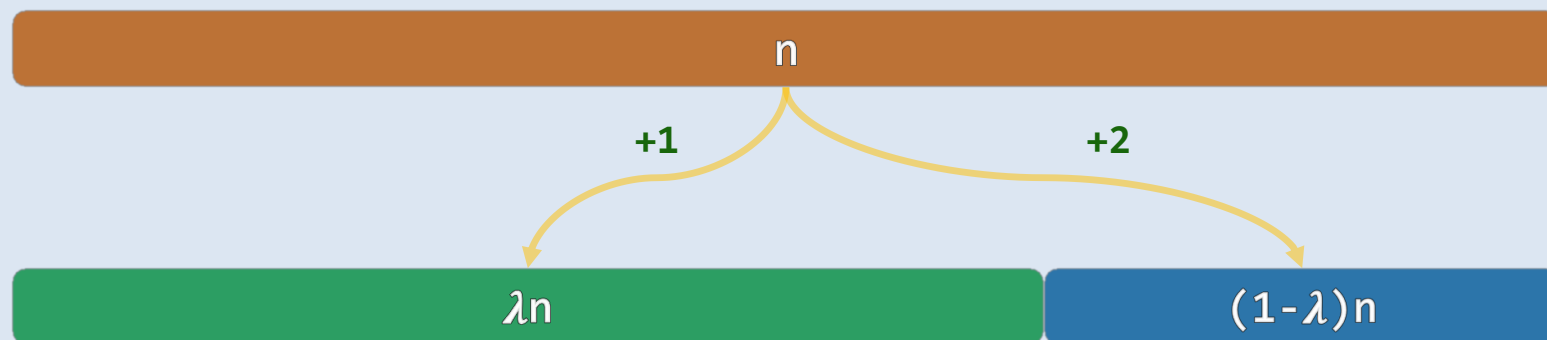


❖ 这类查找算法的渐近复杂度为 $\alpha(\lambda) \cdot \log_2 n = \mathcal{O}(\log n)$

❖ (在常系数意义上的) 性能最优，即意味着 $\alpha(\lambda)$ 达到最小

$$\phi = 0.6180339\dots$$

❖ 递推式: $\alpha(\lambda) \cdot \log_2 n = \lambda \cdot \underline{[1 + \alpha(\lambda) \cdot \log_2 (\lambda n)]} + (1 - \lambda) \cdot \underline{[2 + \alpha(\lambda) \cdot \log_2 ((1 - \lambda)n)]}$



❖ 整理后: $\frac{-\ln 2}{\alpha(\lambda)} = \frac{\lambda \cdot \ln \lambda + (1 - \lambda) \cdot \ln(1 - \lambda)}{2 - \lambda}$

❖ 当 $\lambda = \phi = (\sqrt{5} - 1)/2$ 时, $\alpha(\lambda) = 1.440420\dots$ 达到最小

向量

有序向量：二分查找（版本B）

改进思路

❖ 二分查找中左、右分支转向代价不平衡的问题，也可**直接**解决，比如...

每次迭代仅做**1次**关键码比较；如此，所有分支只有**2个**方向，而不再是**3个**

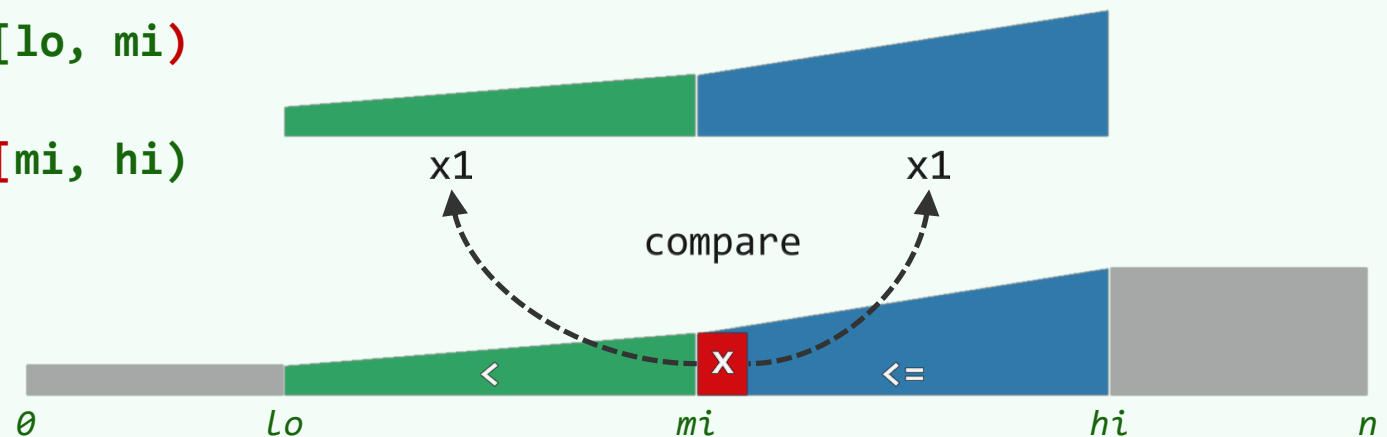
❖ 同样地，轴点 mi 取作中点，则查找每深入一层，问题规模也缩减一半

- $e < x$: 则深入左侧的 $[lo, mi)$

- $x \leq e$: 则深入右侧的 $[mi, hi)$

❖ 直到 $hi - lo = 1$

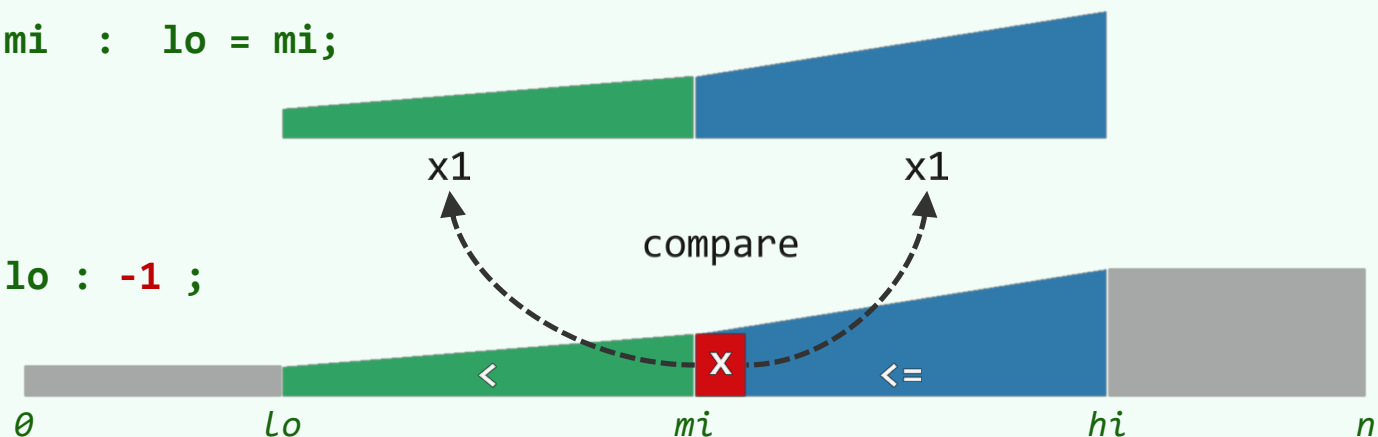
才判断是否命中



❖ 相对于版本A，最好（坏）情况下更坏（好），整体性能更趋均衡

实现

```
❖ template <typename T> static Rank binSearch( T * S, T const & e, Rank lo, Rank hi ) {  
    while ( 1 < hi - lo ) { //有效查找区间的宽度缩短至1时, 算法才终止  
        Rank mi = (lo + hi) >> 1; //以中点为轴点, 经比较后确定深入[lo, mi)或[mi, hi)  
        e < S[mi] ? hi = mi : lo = mi;  
    } //出口时hi = lo + 1  
    return e == S[lo] ? lo : -1 ;  
}
```



❖ 返回命中元素的秩, 或者 (失败时) 非法的秩

返回更多信息

❖ 各种特殊情况，如何统一地处置？比如

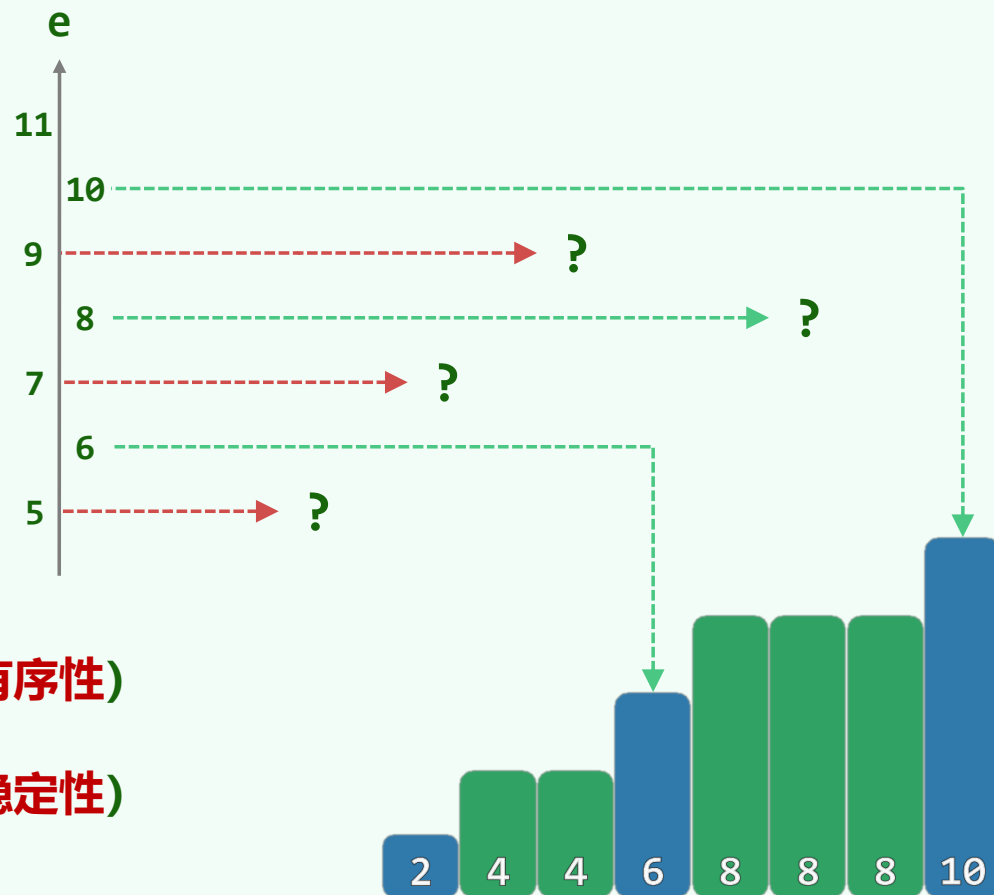
- 目标元素**不存在**；或反过来
- 目标元素同时存在**多个**

❖ 有序向量自身，如何便捷地维护？

比如：`V.insert( 1 + V.search(e), e )`

- 即便失败，也应给出新元素可安置的位置（**有序性**）
- 若有重复元素，也需按其插入的次序排列（**稳定性**）

❖ 为此，需要更为精细、明确、简捷地定义search()的返回值



返回值语义的扩充

❖ 约定: search(**e**)总是返回

m = 不大于**e**的最后一个元素 (或 $-\infty$)

其后继 **M** = 大于**e**的第一个元素 (或 $+\infty$)

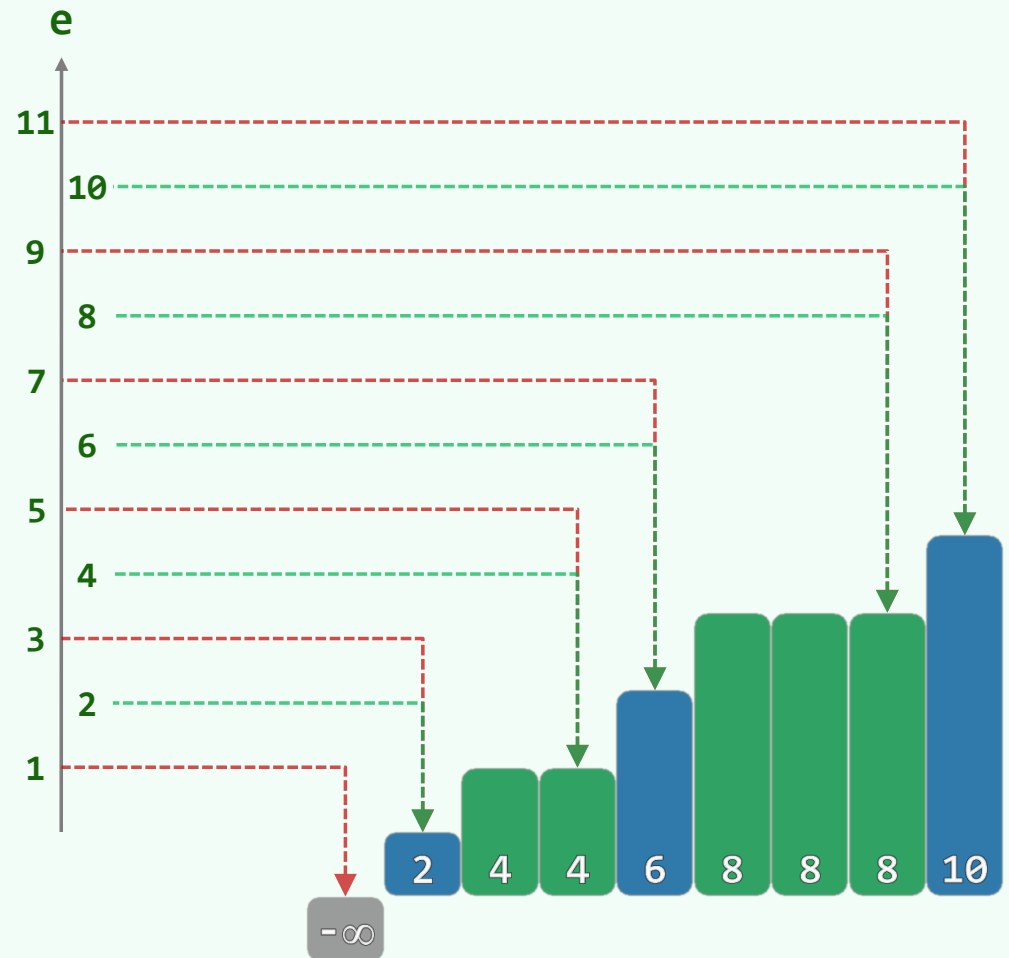
❖ 改进版本B:

```
return e == S[lo] ? lo : -1 ;
```

```
return e < S[lo] ? lo-1 : lo ;
```

❖ 虽可行, 但不免有些蹩脚

有没有...更为...简明、高明的...实现方式?



向量

有序向量：二分查找（版本C）

实现

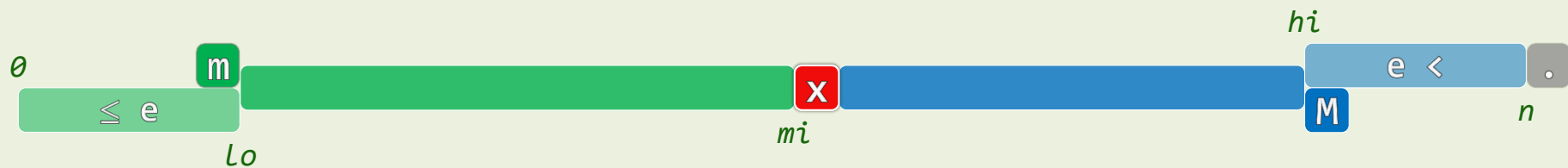
```
template <typename T> static Rank binSearch( T * S, T const & e, Rank lo, Rank hi ) {  
    while ( lo < hi ) { //不变性:  $A[0, lo) \leq e < A[hi, n)$   
        Rank mi = (lo + hi) >> 1;  
  
        e < S[mi] ? hi = mi : lo = mi + 1; // $[lo, mi)$ 或 $(mi, hi)$   
    } //出口时, 必有 $S[lo = hi] = M$   
    return lo - 1; //故,  $S[lo-1] = m$   
} //留意与版本B的差异...
```

❖ 待查找区间宽度缩短至0而非1时, 算法才结束 **//lo == hi**

转入右侧子向量时, 左边界取作mi+1而非mi **// $A[mi]$ 会被遗漏?**

无论成功与否, 返回的秩严格符合接口的语义约定... **//如何证明?**

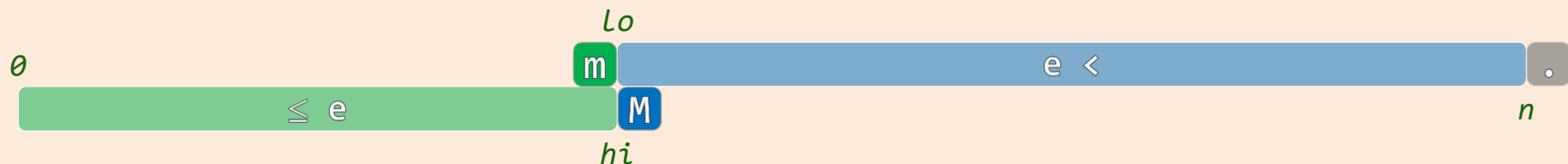
Loop Invariant: $A[0, lo) \leq e < A[hi, n)$



❖ 在算法执行过程中的任意时刻

- $A[lo-1]$ 总是（截至当前已确认的）不大于 e 的最大者 (m)
- $A[hi]$ 总是（截至当前已确认的）大于 e 的最小者 (M)

❖ 当算法终止时, $A[lo-1] = A[hi-1]$ 即是（全局）不大于 e 的最大者



Loop Invariant: $A[0, lo) \leq e < A[hi, n)$

❖ 初始时, $lo = 0$ 且 $hi = n$, $A[0, lo) = A[hi, n) = \emptyset$, 自然成立

❖ 数学归纳: 假设不变性一直保持至(a); 以下无非两种情况...

(b) $\leq e$ $e <$.

(a) $\leq e$ $e <$.
 0 lo mi hi n
1 x compare
compare x 1

(c) $\leq e$ $e <$.

向量
可扩充向量：算法

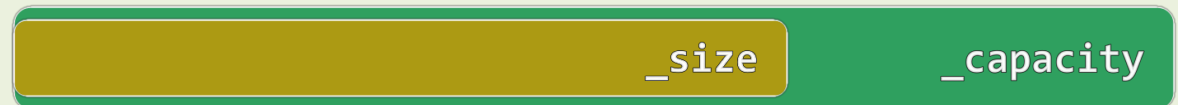
静态空间管理

❖ 开辟内部数组 `_elem[]` 并使用一段地址连续的物理空间

`_capacity`: 总容量

`_elem`

`_size`: 当前的实际规模 n



❖ 若采用静态空间管理策略，容量 `_capacity` 固定，则有明显的不足...

- 上溢/overflow: `_elem[]` 不足以存放所有元素，尽管此时系统往往仍有足够的空间
- 下溢/underflow: `_elem[]` 中的元素寥寥无几

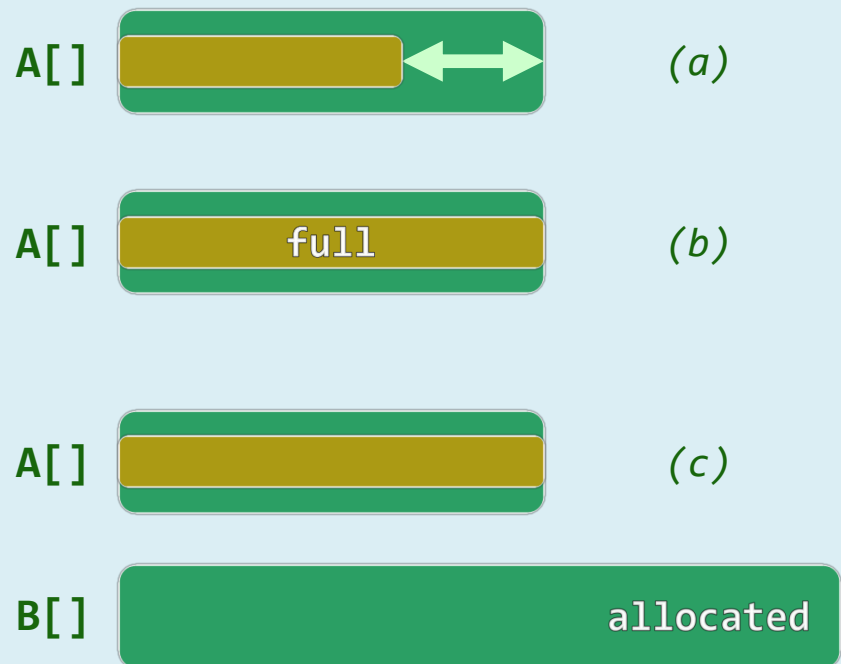
装填因子/load factor: $\lambda = \text{_size} / \text{_capacity} \ll 50\%$

❖ 一般的应用环境中，难以准确预测空间的需求量

❖ 可否使得向量可随实际需求动态调整容量，并同时保证高效率？

动态空间管理

- ❖ 蝉：身体经过一段时间的生长，会蜕去原先的外壳，代之以**更大**的新外壳
- ❖ 向量：在即将**上溢**时，**适当扩大**内部数组的容量



扩容算法

```
❖ template <typename T> void Vector<T>::expand() { //向量空间不足时扩容

    if ( _size < _capacity ) return; //尚未满员时, 不必扩容

    _capacity = max( _capacity, DEFAULT_CAPACITY ); //不低于最小容量

    T* oldElem = _elem; _elem = new T[ _capacity <<= 1 ]; //容量加倍

    for ( Rank i = 0; i < _size; i++ ) //复制原向量内容

        _elem[i] = oldElem[i]; //T为基本类型, 或已重载赋值操作符 '='

    delete [] oldElem; //释放原空间

} //得益于向量的封装, 尽管扩容之后数据区的物理地址有所改变, 却不致出现野指针

❖ 为何必须采用容量加倍策略呢? 其它策略是否可行?
```

向量
可扩充向量：分摊

容量递增策略

❖ `T* oldElem = _elem; _elem = new T[ _capacity += INCREMENT ];` //追加固定增量

❖ 最坏情况：在初始容量 θ 的空向量中，连续插入 $n = m \cdot I \gg 2$ 个元素，而无删除操作

❖ 于是，在第 $1, I + 1, 2I + 1, 3I + 1, 4I + 1, \dots$ 次插入时，都需扩容

$(m-1) \cdot I$ $m * I = O(n)$

...

...

$2 \cdot I$ $3 * I$

I $2 * I$

θ Increment

❖ 即便不计申请空间操作，各次扩容过程中复制原向量的时间成本依次为

$0, I, 2I, 3I, 4I, \dots (m-1) \cdot I$ //算术级数

总体耗时 = $O(n^2)$ ，每次 (insert/remove) 操作的分摊成本为 $O(n)$

容量加倍策略

❖ `T* oldElem = _elem; _elem = new T[ _capacity <<= 1 ];` //容量加倍

❖ 最坏情况：在初始容量1的满向量中，连续插入 $n = 2^m \gg 2$ 个元素，而无删除操作

❖ 于是，在第 1、2、4、8、16、... 次插入时，都需扩容

$2^{(m-1)}$

$2^m = O(n)$

...

.....

4

8

2

4

1

2

0

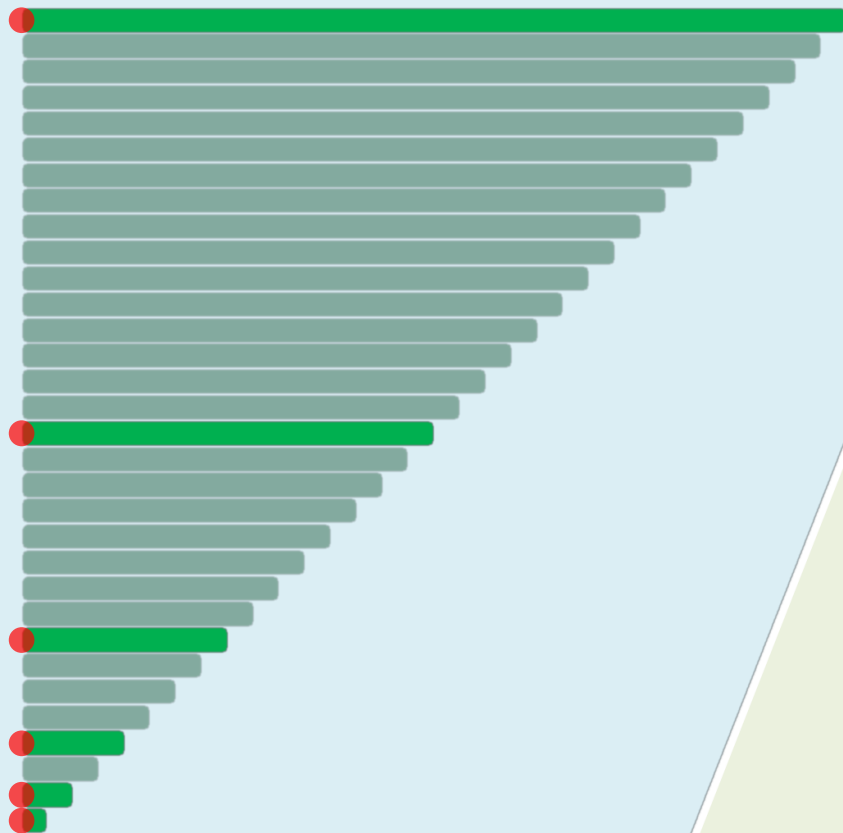
1

❖ 各次扩容过程中复制原向量的时间成本依次为

1, 2, 4, 8, 16, ... 2^{m-1} , $2^m = n$ //几何级数

总体耗时 = $O(n)$, 每次 (insert/remove) 操作的分摊成本为 $O(1)$

对比



	递增策略	倍增策略
累计扩容时间	$O(n^2)$	$O(n)$
分摊扩容时间	$O(n)$	$O(1)$
空间利用率 (装填因子)	$\approx 100\%$	$> 50\%$

平均分析 vs. 分摊分析

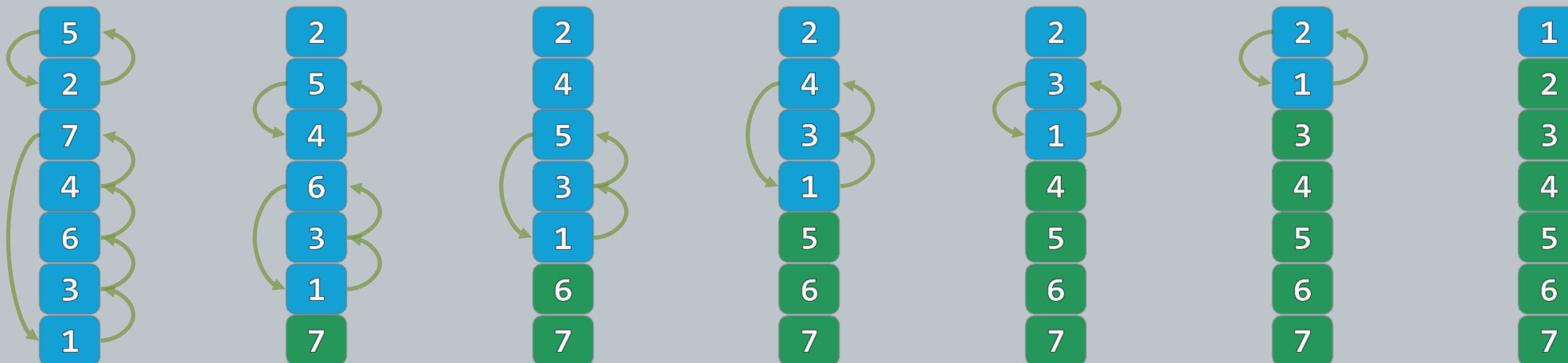
- ❖ **平均** (average complexity) : 根据各种操作出现概率的分布, 将对应的成本加权平均
 - 各种可能的操作, 作为**独立**事件分别考查
 - 割裂了操作之间的**相关性**和**连贯性**
 - 往往**不能准确**地评判数据结构和算法的真实性能
- ❖ **分摊** (amortized complexity) : **连续实施的足够多次操作**, 所需**总体成本**摊还至**单次操作**
 - 从实际可行的角度, 对一系列操作做整体的考量
 - 更加**忠实**地刻画了可能出现的操作序列
 - 更为**精准**地评判数据结构和算法的真实性能
- ❖ 后面将看到更多、更复杂的例子

向量 起泡排序

排序器：统一入口

```
❖ template <typename T> void Vector<T>::sort( Rank lo, Rank hi ) {  
    switch ( rand() % 6 ) {  
        case 1: bubbleSort( lo, hi ); break; //起泡排序  
        case 2: selectionSort( lo, hi ); break; //选择排序 (习题)  
        case 3: mergeSort( lo, hi ); break; //归并排序  
        case 4: heapSort( lo, hi ); break; //堆排序 (第12章)  
        case 5: quickSort( lo, hi ); break; //快速排序 (第14章)  
        default: shellSort( lo, hi ); break; //希尔排序 (第14章)  
    } //随机选择算法，以尽可能充分地测试。应用时可视具体问题的特点，灵活确定或扩充  
}
```

构思

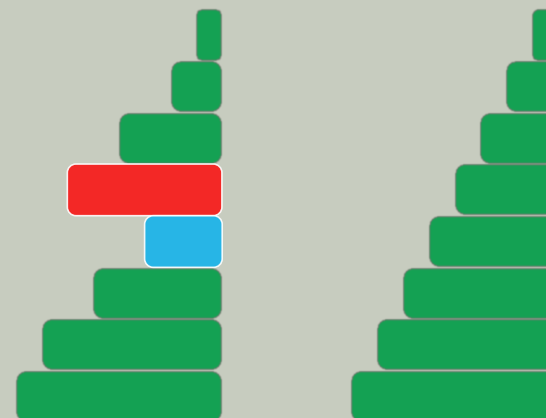


❖ 问题：给定 n 个**可比较**的元素，将它们按（非降）序排列

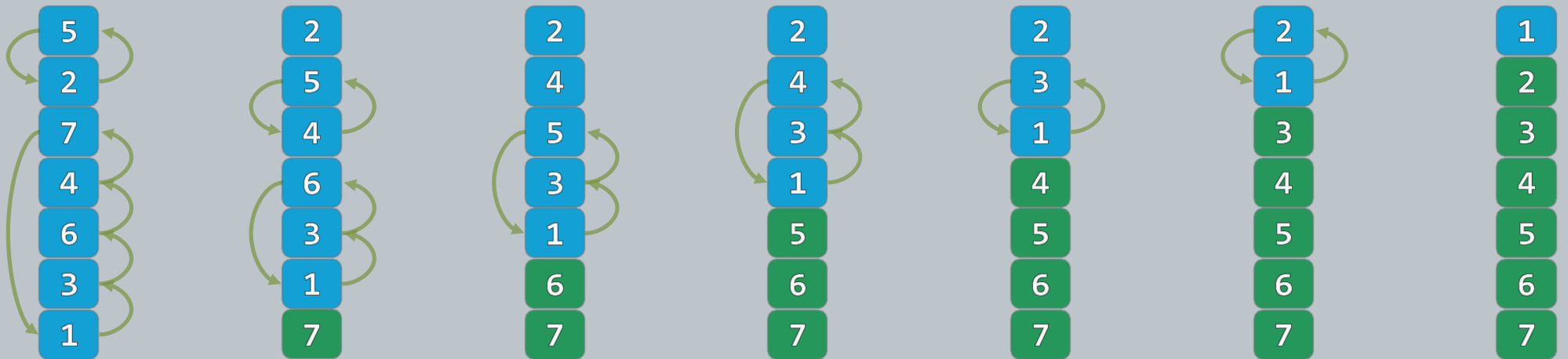
❖ 观察：有序/无序序列中，任何/总有一对**相邻**元素顺序/逆序

❖ 扫描交换：依次比较每一对相邻元素；如有必要，交换之

❖ 若整趟扫描都没有进行交换，则排序完成；否则，再做一趟扫描交换



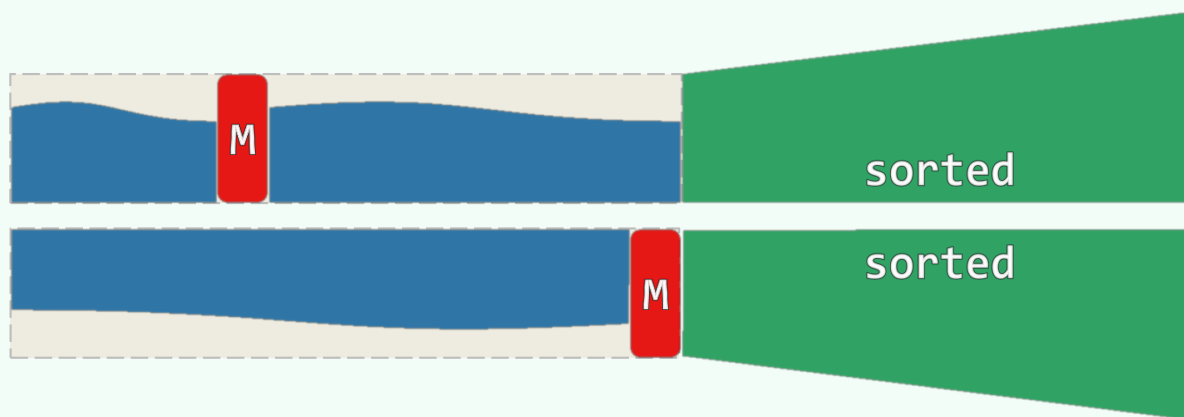
基本版



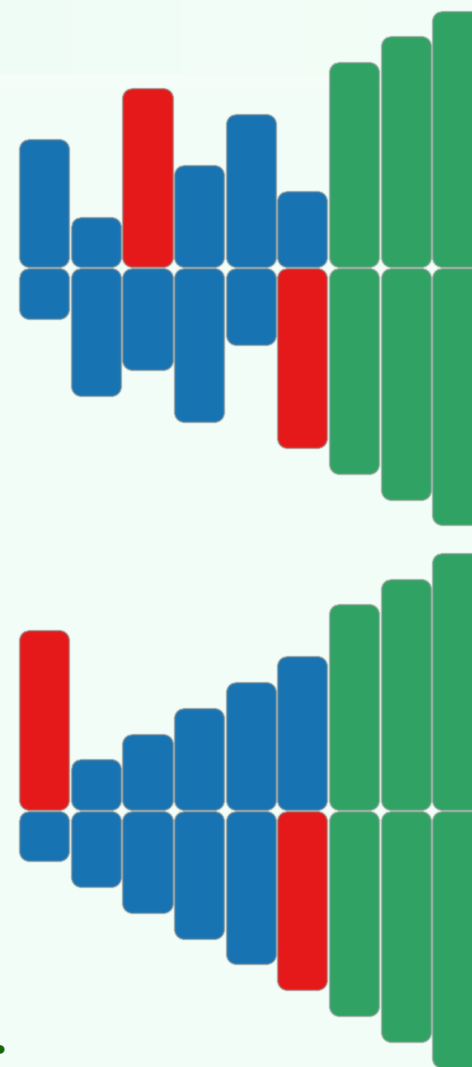
```
❖ template <typename T> Vector<T>::bubbleSort( Rank lo, Rank hi ) {  
    while( lo < --hi ) //逐趟起泡扫描 (输入保证: 0 <= lo < hi <= size)  
        for( Rank i = lo; i < hi; i++ ) //若相邻元素  
            if( _elem[i] > _elem[i + 1] ) //逆序  
                swap( _elem[i], _elem[i + 1] ); //则交换  
}
```


正确性

- ❖ **Loop Invariant**: 经 k 趟扫描交换后, 最大的 k 个元素必然就位
- ❖ **Convergence**: 经 k 趟扫描交换后, 问题规模缩减至 $n-k$
- ❖ **Correctness**: 经至多 n 趟扫描后, 算法必然终止, 且能给出正确解答



- ❖ $n-1$ 趟起泡扫描一定**足够**, 但往往**不必**, 比如...
- ❖ $[hi]$ 就位后, $[lo, hi)$ 可能已经有序 (**sorted**) ——此时, 应该可以...

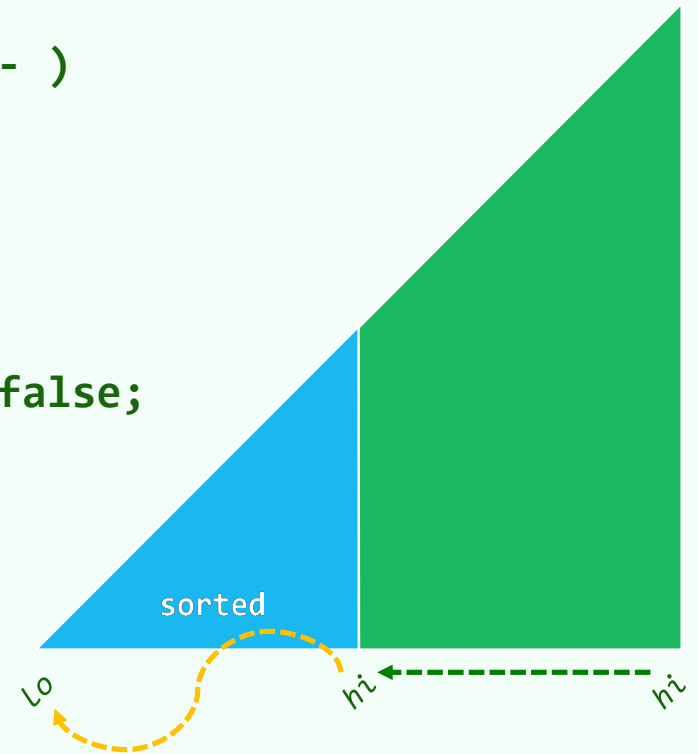


提前终止版

```
template <typename T> void Vector<T>::bubbleSort( Rank lo, Rank hi ) {  
    for ( bool sorted = false; sorted = !sorted; hi-- )  
        for ( Rank i = lo + 1; i < hi; i++ )  
            if ( _elem[i-1] > _elem[i] )  
                swap( _elem[i-1], _elem[i] ), sorted = false;  
}
```

❖ 有改进，但仍有继续改进的余地，比如...

有可能某一后缀[**last**, hi)已然有序——此时，应该可以...

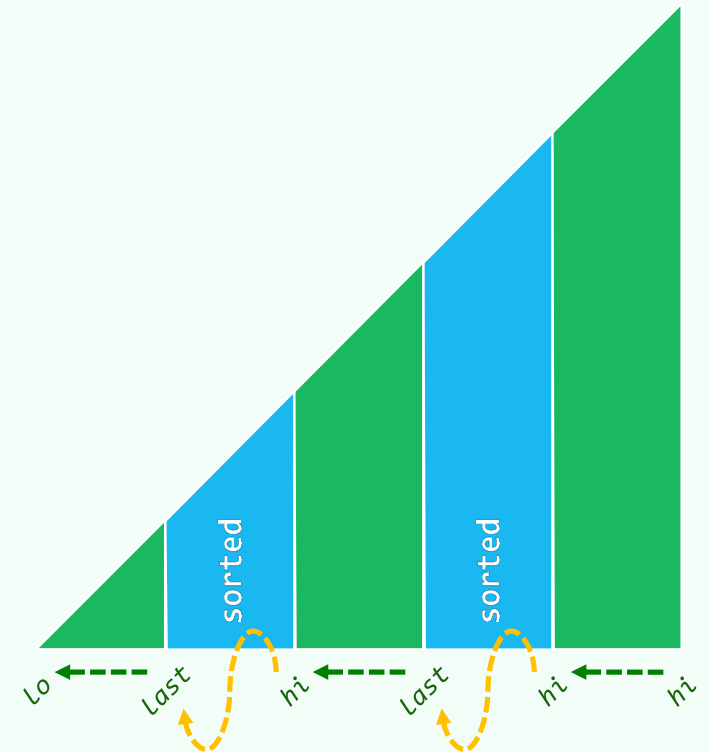


跳跃版

```
template <typename T> void Vector<T>::bubbleSort( Rank lo, Rank hi ) {  
    for ( Rank last; lo < hi; hi = last )  
        for ( Rank i = (last = lo) + 1; i < hi; i++ )  
            if ( _elem[i-1] > _elem[i] )  
                swap( _elem[i-1], _elem[i] ), last = i;  
}
```

❖ $A[lo, \text{last}) \leq A[\text{last}, hi)$

$A[\text{last} - 1] < A[\text{last}, hi)$



综合评价

- ❖ 时间效率：最好 $O(n)$ ，最坏 $O(n^2)$
- ❖ 输入含重复元素时，算法的**稳定性** (stability) 是更为细致的要求
重复元素在输入、输出序列中的相对次序，是否保持不变？
 - 输入： 6, 7a, 3, 2, 7b, 1, 5, 8, 7c, 4
 - 输出： 1, 2, 3, 4, 5, 6, 7a, 7b, 7c, 8 //stable
1, 2, 3, 4, 5, 6, 7a, 7c, 7b, 8 //unstable
- ❖ 以上起泡排序算法是稳定的吗？
是的！毕竟在起泡排序中，**唯有相邻**元素才可交换
- ❖ 在if一句的判断条件中，若把">"换成">="，将有何变化？

归并排序：分而治之

原理

❖ //分治策略

//向量与列表通用

//J. von Neumann, 1945



序列一分为二 // $O(1)$

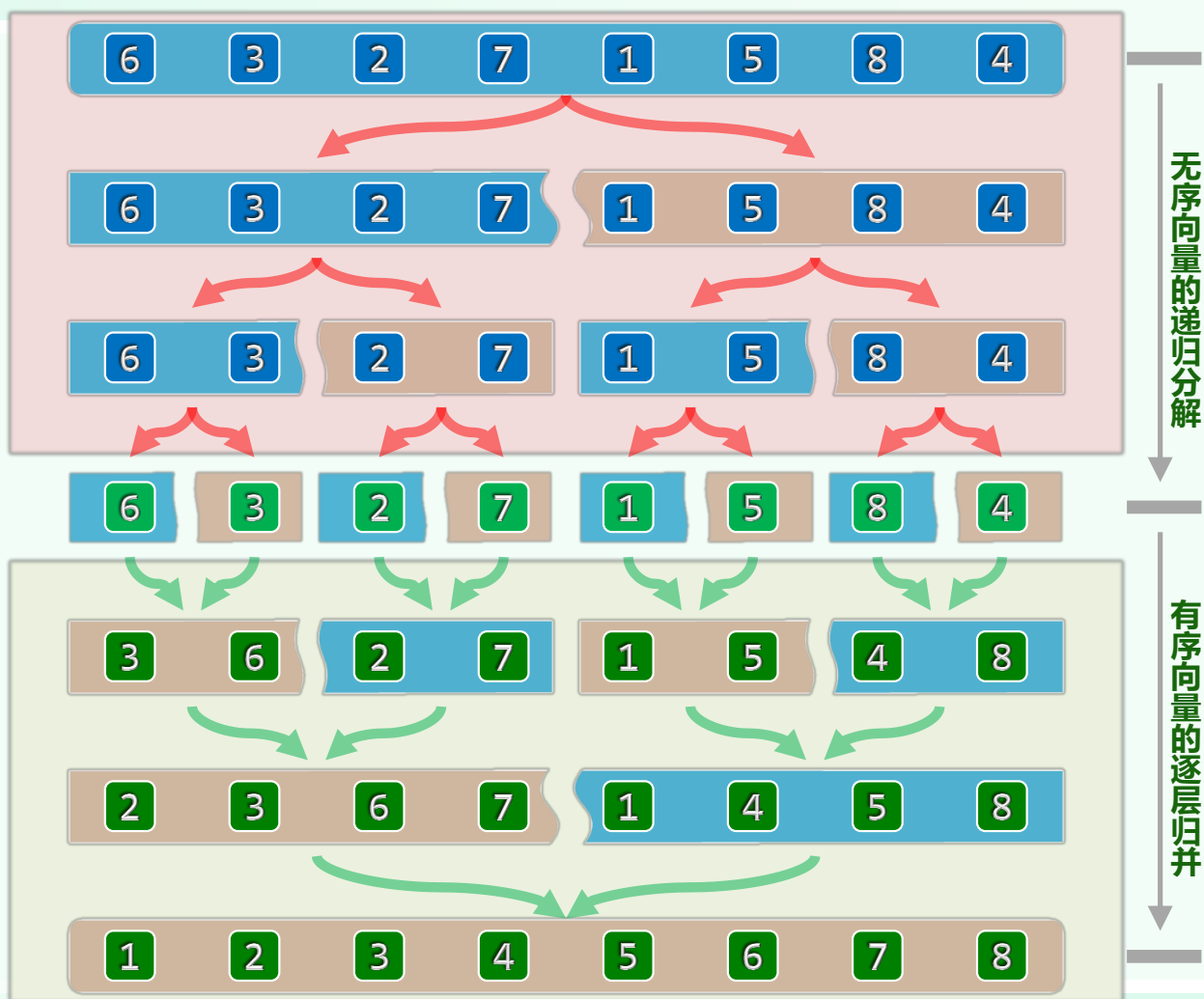
子序列递归排序 // $2 \times T(n/2)$

合并有序子序列 // $O(n)$

如何递推？

❖ 若真能如此，整体的运行成本

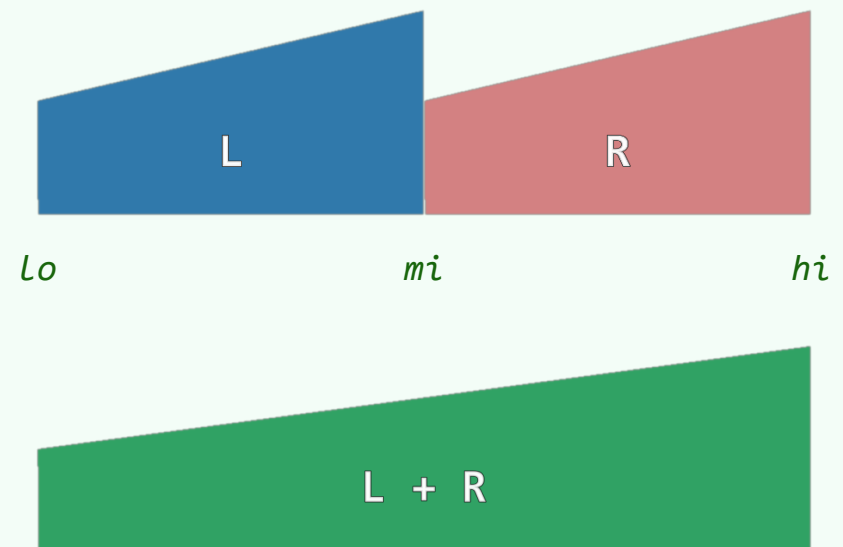
应是 $O(n \cdot \log n)$



分而治之

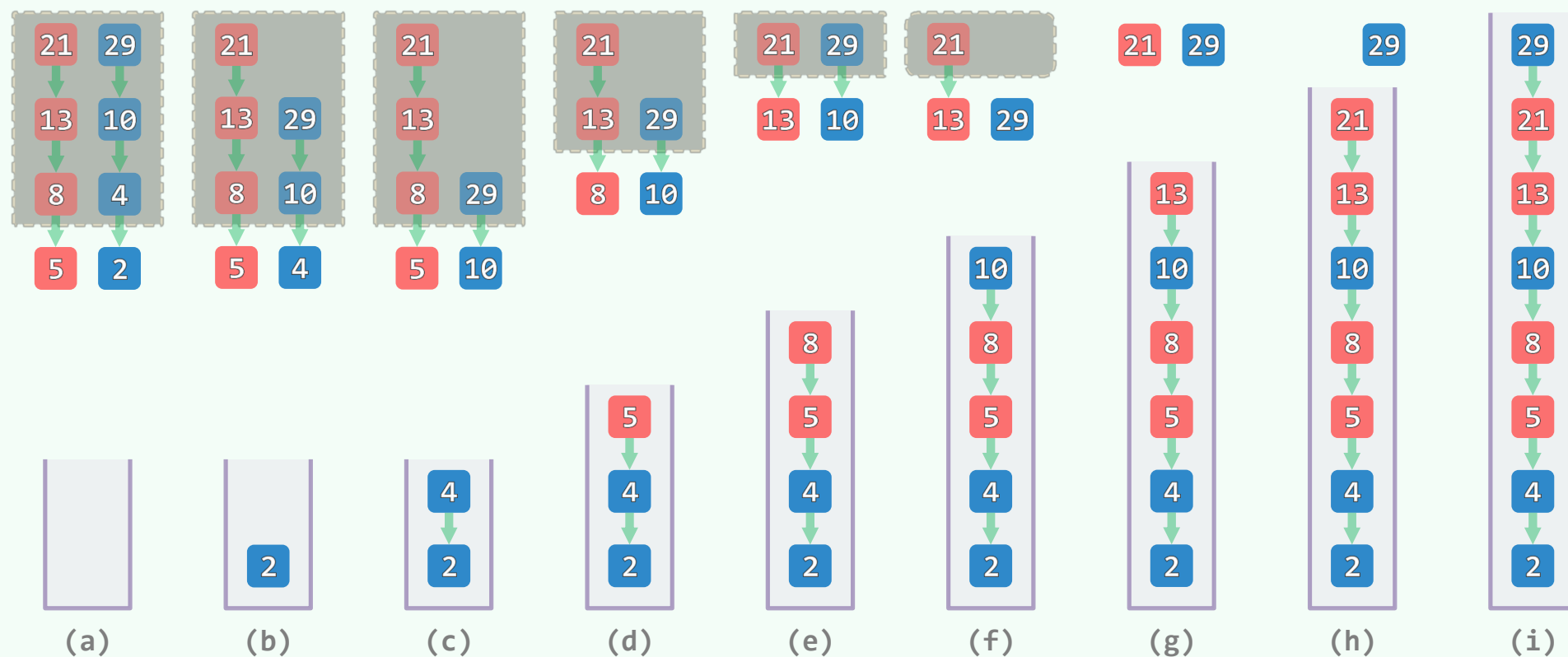
```
❖ template <typename T> void Vector<T>::mergeSort( Rank lo, Rank hi ) {  
  
    if ( hi - lo < 2 ) return; //单元素区间自然有序, 否则...  
  
    int mi = (lo + hi) >> 1; //以中点为界  
  
    mergeSort( lo, mi ); //对前半段排序  
  
    mergeSort( mi, hi ); //对后半段排序  
  
    merge( lo, mi, hi ); //归并  
  
}
```

如果子序列已经有序, 是否可能改进?



二路归并

❖ **2-way merge**: 有序序列, 合二为一, 保持有序: $S[lo, hi) = S[lo, mi) + S[mi, hi)$



实现 (1/2) : 预备

```
template <typename T> //[lo, mi)和[mi, hi)各自有序
```

```
void Vector<T>::merge( Rank lo, Rank mi, Rank hi ) { //lo < mi < hi
```

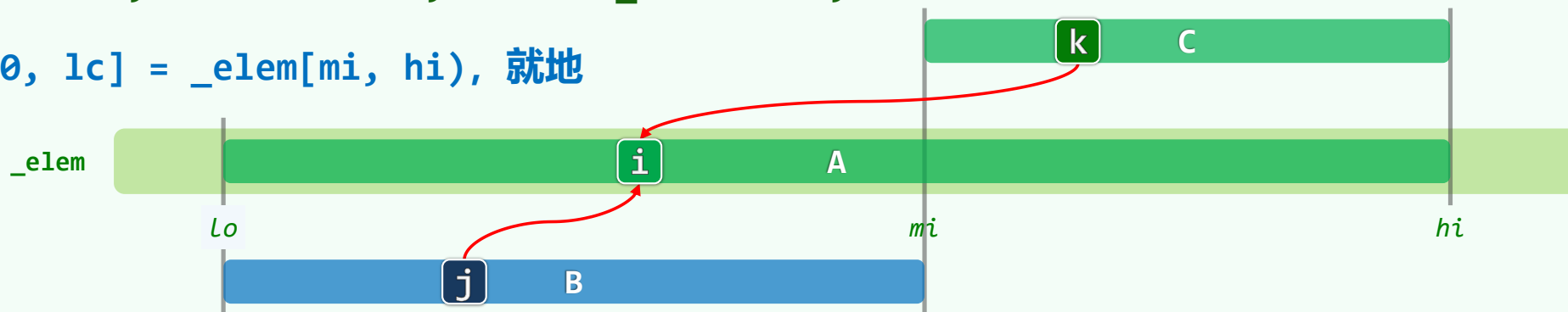
```
    Rank i = 0; T* A = _elem + lo; //A = _elem[lo, hi), 就地
```

```
    Rank j = 0, lb = mi - lo; T* B = new T[lb]; //B[0, lb) <-- _elem[lo, mi)
```

```
    for ( Rank i = 0; i < lb; i++ ) B[i] = A[i]; //复制自A的前缀
```

```
    Rank k = 0, lc = hi - mi; T* C = _elem + mi;
```

```
    //C[0, lc] = _elem[mi, hi), 就地
```



实现 (2/2) : 归并

```
while ( ( j < lb ) && ( k < lc ) ) //反复地比较B、C的首元素
```

```
    A[i++] = ( B[j] <= C[k] ) ? B[j++] : C[k++]; //将更小者归入A中
```

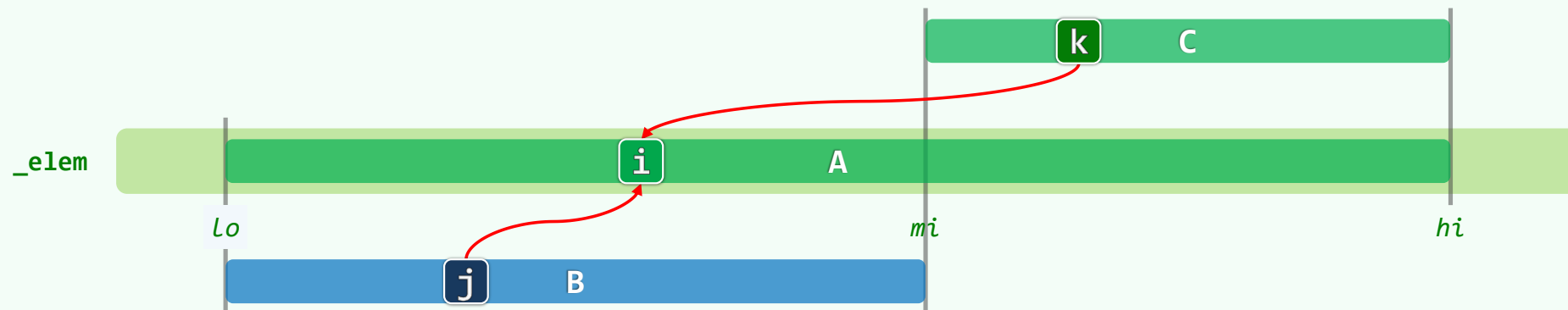
```
while ( j < lb ) //若C先耗尽, 则
```

改成 $B[j] < C[k]$ 会怎么样?

```
    A[i++] = B[j++]; //将B残余的后缀归入A中——若B先耗尽呢?
```

```
delete [] B; //new和delete非常耗时, 如何减少?
```

```
}
```

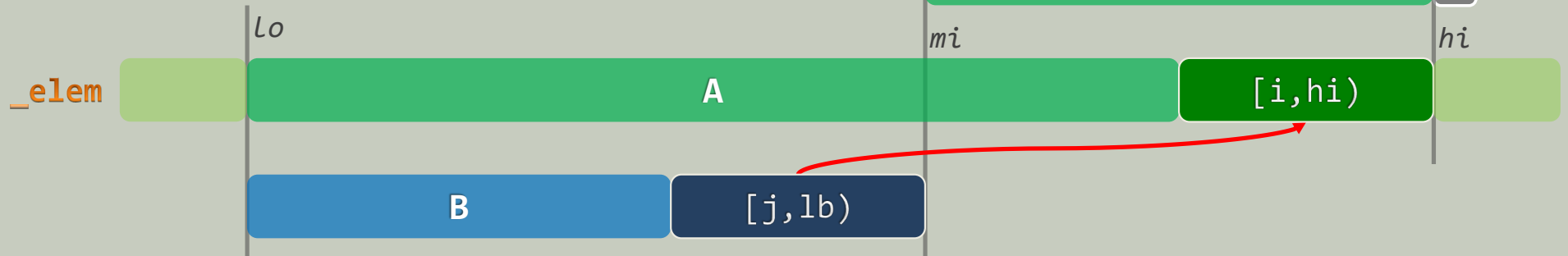


正确性 & 效率

(a)



(b)



运行时间

❖ 二路归并中，两个while循环每迭代一步i都会递增；j或k中之一也会随之递增

❖ 因： $0 \leq j \leq lb$, $0 \leq k \leq lc$

故： 累计迭代步数 $\leq lb + lc = n$

二路归并只需 $\mathcal{O}(n)$ 时间

(注意，即便lb和lc不相等，甚至相差悬殊，这一结论依然成立)

❖ 于是可知，归并排序的时间复杂度为 $\mathcal{O}(n \log n)$

❖ 这一算法的思路及结论，也适用于另一类序列——列表（下一章）

```
while ( ( j < lb ) && ( k < lc ) )  
    A[i++] = ( B[j] <= C[k] ) ? B[j++] : C[k++];  
while ( j < lb )  
    A[i++] = B[j++];
```